Algorithm Design

Notes

Contents

Chapter 1

Algorithm Analysis

1.1 Computational Tractability

The question of how to solve a problem in the *shortest possible time* is not new. A simple approach to many problems is **brute force**: try every possible solution and pick the best one. The problem is that this usually takes 2^n time or worse, where n is the size of the input. For large inputs, this is completely impractical.

1.1.1 Polynomial Running Time

We say that an algorithm is **efficient** if its running time is *polynomial*.

Definition 1 (Polynomial-time algorithm)**.** *An algorithm runs in polynomial time if there exist constants $c > 0$ and $d > 0$ such that, on every input of size n, its running time is at most $c \cdot n^d$ steps.*

A useful way to think about this: if you *double* the input size, the running time should only grow by a constant factor $C = 2^d$.

(To represent the number x we need $\log_2 x$ bits, so the input size is $n = \log_2 x$. If the algorithm runs in polynomial time, then the running time is $O(n^d) = O((\log_2 x)^d) = O(\log_2^d x)$)

The input change for combinatorial problems and numerical problems is different. For combinatorial problems, the input size is typically the number of elements (e.g., nodes in a graph), while for numerical problems, the input size is the number of bits needed to represent the numbers.

Why polynomial = efficient? In practice, poly-time algorithms tend to have small constants and small exponents. Also, breaking the exponential barrier usually means discovering an important structure in the problem.

Exceptions. A few poly-time algorithms have very large constants and are not useful in practice. On the other hand, some exponential-time algorithms (like the Simplex method) are widely used because the worst cases almost never happen in real life.

1.1.2 Types of Analysis

There are several ways to measure the running time of an algorithm:

Type	Description	Example
Worst-case	Guarantee for *any* input of size n	Heapsort: $\leq 2n \log_2 n$ compares
Probabilistic	Expected time of a randomised algorithm	Quicksort: $\sim 2n \ln n$ compares
Amortised	Worst-case over a sequence of n operations	Stack resize: $O(n)$ for n push/pop
Average-case	Expected time on a random input	3-way radix sort: $\sim 2n \ln n$

Worst-case analysis is the most common. It gives a solid guarantee for all possible inputs and usually reflects real performance well.

Sometimes the algorithms aren't deterministic, so we use **probabilistic analysis** to measure the expected running time. This is common for randomised algorithms, which use random bits to make decisions.In this case, we use the expexted value of the running time over the random choices of the algorithm.

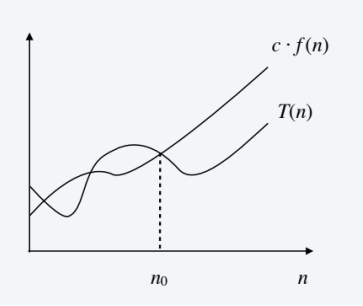
1.2 Asymptotic Order of Growth

Asymptotic notation lets us describe how the running time *grows* as n gets large, ignoring small constant factors.

1.2.1 Big-O Notation (Upper Bound)

Definition 2 (Big-O). *$T(n)$ is $O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that*

$$T(n) \leq c \cdot f(n) \quad \textit{for all } n \geq n_0.$$



An equivalent condition using limits:

$$\limsup_{n \to \infty} \frac{T(n)}{f(n)} < \infty.$$

Example. Let $T(n) = 32n^2 + 17n + 1$. Then:

- $T(n)$ is $O(n^2)$ (choose $c = 50$, $n_0 = 1$)
- $T(n)$ is also $O(n^3)$
- $T(n)$ is *not* $O(n)$ nor $O(n \log n)$

Typical use. "Insertion sort makes $O(n^2)$ compares to sort n elements."

Note on notation. Technically, $O(f(n))$ is a *set* of functions, but computer scientists write $T(n) = O(f(n))$ as shorthand. This is an accepted abuse of notation — just be careful not to misuse it (e.g., writing $O(n) = O(n^2)$ is fine, but not the reverse).

1.2.2 Big-Omega Notation (Lower Bound)

Definition 3 (Big-Omega)**.** *$T(n)$ is $\Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that*

$$T(n) \geq c \cdot f(n) \quad \textit{for all } n \geq n_0.$$

Example. For $T(n) = 32n^2 + 17n + 1$:

- $T(n)$ is $\Omega(n^2)$ and $\Omega(n)$
- $T(n)$ is *not* $\Omega(n^3)$

Typical use. "Any comparison-based sorting algorithm requires $\Omega(n \log n)$ compares in the worst case."

Common mistake. It is meaningless to say "requires *at least* $O(n \log n)$ compares" — Big-O is an upper bound, not a lower bound.

1.2.3 Big-Theta Notation (Tight Bound)

Definition 4 (Big-Theta)**.** *$T(n)$ is $\Theta(f(n))$ if there exist constants $c_1 > 0$, $c_2 > 0$, and $n_0 \geq 0$ such that*

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n) \quad \textit{for all } n \geq n_0.$$

In other words, $T(n)$ is $\Theta(f(n))$ if and only if it is both $O(f(n))$ and $\Omega(f(n))$.

Example. Mergesort makes $\Theta(n \log n)$ compares. For $T(n) = 32n^2 + 17n + 1$: $T(n)$ is $\Theta(n^2)$, but not $\Theta(n)$ or $\Theta(n^3)$.

1.2.4 Useful Limit Rules

Proposition 1. *If $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = c > 0$, then $f(n)$ is $\Theta(g(n))$.*

Proposition 2. *If $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = 0$, then $f(n)$ is $O(g(n))$ but not $\Omega(g(n))$.*

Proof sketch for the first proposition. By definition of the limit, there exists n_0 such that for all $n \geq n_0$:

$$\frac{c}{2} < \frac{f(n)}{g(n)} < 2c \quad \Rightarrow \quad \frac{c}{2} \cdot g(n) \leq f(n) \leq 2c \cdot g(n).$$

This gives both the upper and lower bound needed for Θ. □

1.2.5 Key Asymptotic Facts

- **Polynomials.** If $T(n) = a_0 + a_1 n + \cdots + a_d n^d$ with $a_d > 0$, then $T(n)$ is $\Theta(n^d)$, because
$$\lim_{n\to\infty} \frac{a_0 + a_1 n + \cdots + a_d n^d}{n^d} = a_d > 0.$$
- **Logarithm base does not matter.** $\Theta(\log_a n) = \Theta(\log_b n)$ for any constants $a, b > 0$ (by change-of-base formula). We simply write $\log n$.

- **Logs are slower than polynomials.** For every $d > 0$, $\log n$ is $O(n^d)$.
- **Polynomials are slower than exponentials.** For every $r > 1$ and $d > 0$, n^d is $O(r^n)$, i.e.,

$$\lim_{n\to\infty} \frac{n^d}{r^n} = 0.$$

1.2.6 Big-O with Multiple Variables

Sometimes the input has two parameters (e.g., nodes and edges in a graph).
Definition 5. *$T(m, n)$ is $O(f(m, n))$ if there exist constants $c > 0$, $m_0 \geq 0$, $n_0 \geq 0$ such that $T(m, n) \leq c \cdot f(m, n)$ for all $n \geq n_0$ and $m \geq m_0$.*

Example. $T(m, n) = 32mn^2 + 17mn + 32n^3$ is $O(mn^2 + n^3)$ and also $O(mn^3)$, but it is *neither* $O(n^3)$ nor $O(mn^2)$.

Typical use. Breadth-first search (BFS) takes $O(m + n)$ time to find the shortest path from s to t in a directed graph with n nodes and m edges.

1.3 Survey of Common Running Times

The table below shows the most common complexity classes, from fastest to slowest.

Complexity	Name	Example
$O(\log n)$	Sublinear / Logarithmic	Binary search
$O(n)$	Linear	Finding max, merging two lists
$O(n \log n)$	Linearithmic	Mergesort, Heapsort
$O(n^2)$	Quadratic	Closest pair (naive)
$O(n^3)$	Cubic	Set disjointness
$O(n^k)$	Polynomial	Independent set of size k
$O(2^n)$	Exponential	Maximum independent set

1.3.1 Logarithmic Time — O(log n)

The running time grows very slowly — doubling the input only adds one step.

Binary search. Given a sorted array A of n numbers, find whether x is in the array.

```
lo = 1, hi = n
while (lo <= hi):
    mid = (lo + hi) / 2
    if   x < A[mid]: hi = mid - 1
    elif x > A[mid]: lo = mid + 1
    else: return YES
return NO
```

Each iteration cuts the search interval in half, so at most $\lceil \log_2 n \rceil$ iterations are needed.

1.3.2 Linear Time — O(n)

The running time grows proportionally to the input size.

Example 1 — Maximum of n numbers.

```
max = a[1]
for i = 2 to n:
    if a[i] > max:
        max = a[i]
```

Example 2 — Merging two sorted lists $A = a_1, \ldots, a_n$ and $B = b_1, \ldots, b_n$:

```
i = 1, j = 1
while (both lists are nonempty):
    if a[i] <= b[j]: append a[i] to output; i++
    else:            append b[j] to output; j++
append the rest of the nonempty list
```

Claim. Merging takes $O(n)$ time. After each compare, the output list grows by one element, so at most $2n - 1$ compares are needed.

1.3.3 Linearithmic Time — O(n log n)

This complexity often appears in divide-and-conquer algorithms.

- **Sorting**: Mergesort and Heapsort both use $O(n \log n)$ compares.
- **Largest empty interval**: Given n timestamps $x_1, \ldots, x_n$ when copies of a file arrive at a server, find the longest gap with no arrivals. *Solution*: sort the timestamps ($O(n \log n)$), then scan in order and track the maximum gap between consecutive timestamps ($O(n)$). Total: $O(n \log n)$.

1.3.4 Quadratic Time — Quadratic Complexity

Typical when you enumerate all *pairs* of elements.

Closest pair of points. Given n points $(x_1, y_1), \ldots, (x_n, y_n)$ in the plane, find the closest pair.

```
min = dist(p1, p2)
for i = 1 to n:
    for j = i+1 to n:
        d = (xi - xj)^2 + (yi - yj)^2
        if d < min: min = d
```

This naive solution is $O(n^2)$. It might seem that $\Omega(n^2)$ is unavoidable, but in Chapter 5 (divide-and-conquer) we will see an $O(n \log n)$ solution.

1.3.5 Cubic Time — Cubic Complexity

Typical when you enumerate all *triples* of elements.

Set disjointness. Given n sets $S_1, \ldots, S_n \subseteq \{1, \ldots, n\}$, is there a disjoint pair?

```
for each set Si:
    for each set Sj (j != i):
        for each element p in Si:
            check if p is in Sj
        if no element of Si is in Sj:
            report Si and Sj disjoint
```

The three nested loops give $O(n^3)$ time.

1.3.6 Polynomial Time — Polynomial Complexity

Useful when k is a small constant, but may be impractical for large k.

Independent set of size k. Given a graph, are there k nodes with no edge between any two of them?

```
for each subset S of k nodes:
    if S is an independent set: report S
```

The number of k-element subsets is:

$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k!} \leq \frac{n^k}{k!}.$$

Each check takes $O(k^2)$ time, so the total is $O(k^2 \cdot n^k/k!) = O(n^k)$. This is polynomial for any fixed k, but already impractical for $k = 17$.

1.3.7 Exponential Time — Exponential Complexity

Maximum independent set. Find the largest set of nodes with no edge between any two.

```
S* = empty
for each subset S of nodes:
    if S is independent and |S| > |S*|:
        S* = S
```

There are 2^n subsets, and each check is $O(n^2)$, so the total is $O(n^2 \cdot 2^n)$. This is only feasible for very small n.

1.4 Hierarchy of Complexities

The following chain summarises how the classes relate:

$$O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(n^k) \subset O(2^n).$$

Each class grows strictly faster than the one on its left. In particular:

- Every logarithm grows slower than every polynomial.

- Every polynomial grows slower than every exponential.

These facts follow directly from the limit rules in Section 2.4.

Chapter 2

Greedy algorithm

2.1 Interval Scheduling

2.1.1 The Problem

Input. A set of n jobs. Job j **starts at time s_j and finishes at time f_j.**

Compatibility. Two jobs are *compatible* if they don't overlap in time.

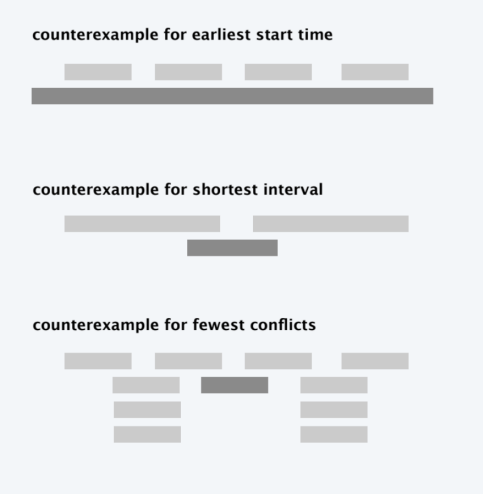
Goal. Find the maximum number of mutually compatible jobs.

2.1.2 Greedy Templates

Several natural greedy rules can be tried:

1. **Earliest start time first:** schedule jobs in ascending order of s_j.
2. **Earliest finish time first:** schedule jobs in ascending order of f_j.
3. **Shortest interval first:** schedule jobs in ascending order of $f_j - s_j$.
4. **Fewest conflicts first:** for each job j, count the number of conflicting jobs c_j, then schedule in ascending order of c_j.

Counterexamples show that only earliest finish time works.



2.1.3 Earliest-Finish-Time-First Algorithm

EARLIEST-FINISH-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT jobs by finish time so that $f_1 \leq f_2 \leq \ldots \leq f_n$

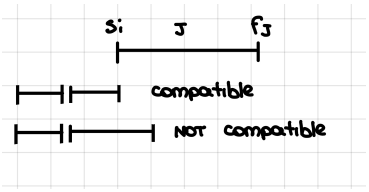
$A \leftarrow \phi$ ⟵ set of jobs selected

FOR $j = 1$ TO n

  IF job j is compatible with A

    $A \leftarrow A \cup \{ j \}$

RETURN A

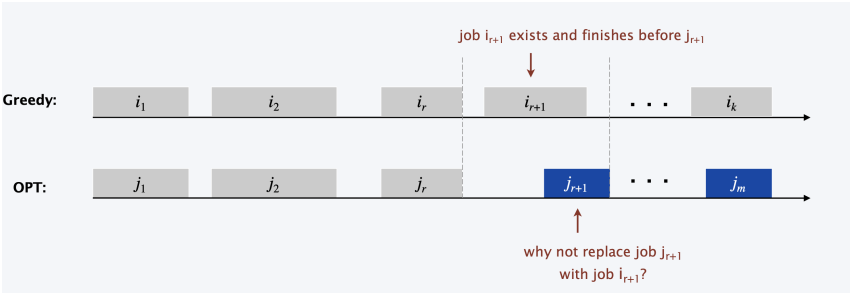


Implementation note. Keep track of the last job j^* added to A. Job j is compatible with A if and only if $s_j \geq f_{j^*}$. Sorting takes $O(n \log n)$ time; the loop is $O(n)$. Total: $O(n \log n)$.

2.1.4 Proof of Optimality

Theorem 1. *The earliest-finish-time-first algorithm is optimal.*

Proof (by contradiction). Assume the greedy algorithm is not optimal. Let $i_1, i_2, \ldots, i_k$ be the jobs selected by the greedy algorithm, and let $j_1, j_2, \ldots, j_m$ be the jobs in an optimal solution with $i_1 = j_1, i_2 = j_2, \ldots, i_r = j_r$ for the largest possible r. (At r, the greedy and optimal solutions first differ.) If $r = k$, then the greedy solution has k jobs and the optimal has $m \geq k$, so they are equal (both optimal). Otherwise, job i_{r+1} exists. Since the greedy algorithm picks jobs by earliest finish time, $f_{i_{r+1}} \leq f_{j_{r+1}}$. We can replace j_{r+1} with i_{r+1} in the optimal solution. This creates a new optimal solution with one more agreement, contradicting the maximality of r.



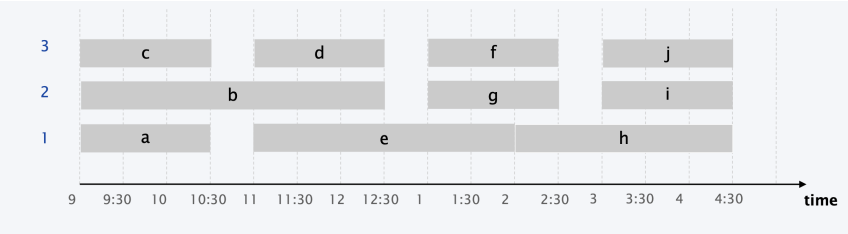
□

2.2 Interval Partitioning

2.2.1 The Problem

Input. A set of n lectures. Lecture j starts at s_j and finishes at f_j.

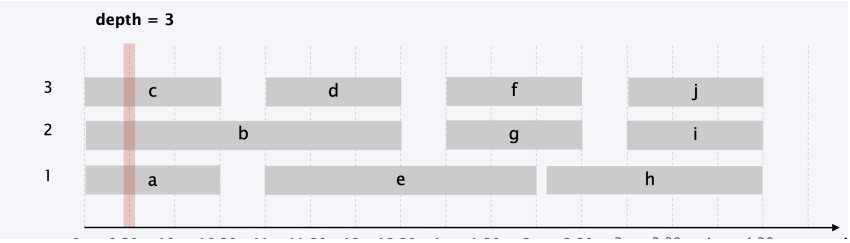
Goal. Schedule all lectures using the *minimum number* of classrooms so that no two lectures overlap in the same classroom.



2.2.2 Depth and Lower Bound

Definition 6 (Depth)**.** *The* depth *of a set of intervals is the maximum number of intervals that contain any given time.*

When we talk about lower bound, we mean a number that is less than or equal to the optimal solution. Since the depth is a lower bound, it means that no schedule can use fewer classrooms than the depth.



Key observation. The number of classrooms needed is at least the depth. The optimal number always equal the depth and the earliest-start-time-first algorithm achieves it.

2.2.3 Earliest-Start-Time-First Algorithm

```
EARLIEST-START-TIME-FIRST (n, s1, s2, ..., sn, f1, f2, ..., fn)

SORT lectures by start time so that s1 ≤ s2 ≤ ... ≤ sn.

d ← 0  ⟵ number of allocated classrooms

FOR j = 1 TO n

    IF lecture j is compatible with some classroom

        Schedule lecture j in any such classroom k.

    ELSE

        Allocate a new classroom d + 1.

        Schedule lecture j in classroom d + 1.

        d ← d + 1

RETURN schedule.
```

Implementation. Use a priority queue to store classrooms, keyed by the finish time of their last lecture. To check if lecture j is compatible, compare s_j to the minimum key in the queue. If compatible, assign j to that classroom and update its key to f_j.
Running time: Sorting takes $O(n \log n)$. Each of the n lectures is processed once, and each priority queue operation takes $O(\log n)$, so the total is $O(n \log n)$.

2.2.4 Proof of Optimality

Theorem 2. *The earliest-start-time-first algorithm is optimal.*

Proof. Let d be the number of classrooms the algorithm uses. Classroom d was opened because we had to schedule some lecture j that was incompatible with all $d-1$ existing classrooms. This means there are d lectures that all overlap at time $s_j + \varepsilon$ (lecture j plus one from each of the other $d-1$ classrooms). Since these d lectures overlap, any schedule must use at least d classrooms. The algorithm uses exactly d classrooms, so it is optimal. □

2.3 Scheduling to Minimize Lateness

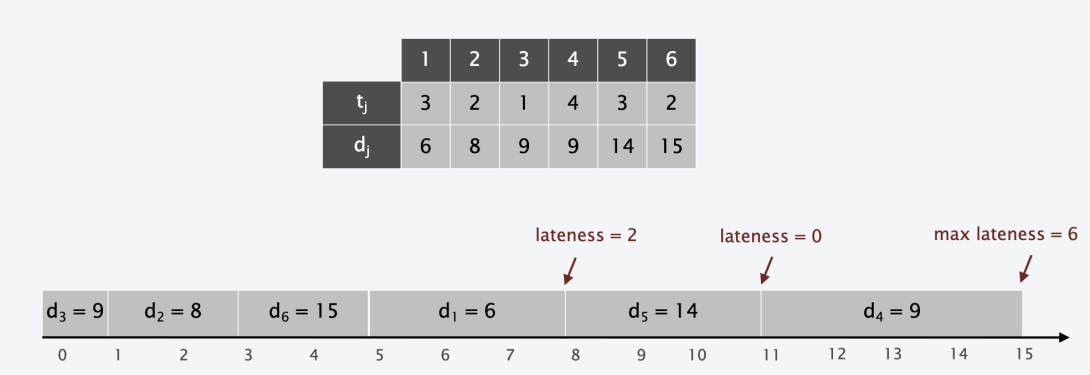
2.3.1 The Problem

Input. A single resource processes one job at a time. Job j requires t_j units of processing time and has a deadline d_j.

Definitions.

- If job j starts at time s_j, it finishes at time $f_j = s_j + t_j$.
- The *lateness* of job j is $\ell_j = \max\{0, f_j - d_j\}$.

Goal. Schedule all jobs to minimize the *maximum lateness* $L = \max_j \ell_j$.



2.3.2 Greedy Candidates

1. **Shortest processing time first:** schedule jobs in ascending order of t_j.
2. **Earliest deadline first:** schedule jobs in ascending order of d_j.
3. **Smallest slack:** schedule jobs in ascending order of $d_j - t_j$ (slack = deadline minus processing time).

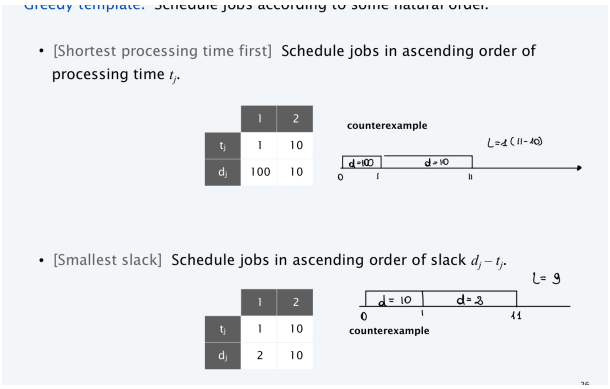
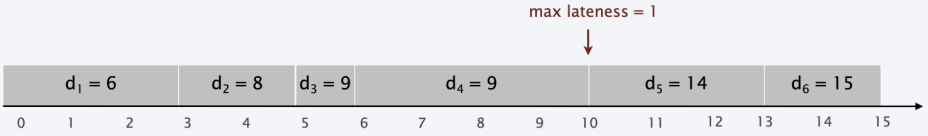


Figure 2.1: Counterexample showing that only earliest-deadline-first works.

2.3.3 Earliest-Deadline-First Algorithm

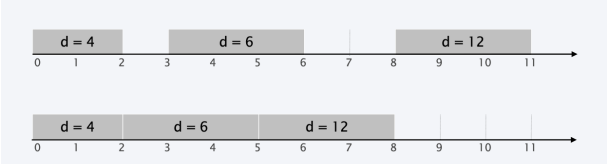
EARLIEST-DEADLINE-FIRST $(n, t_1, t_2, \ldots, t_n, d_1, d_2, \ldots, d_n)$

SORT n jobs so that $d_1 \leq d_2 \leq \ldots \leq d_n$.
$t \leftarrow 0$
FOR $j = 1$ TO n
  Assign job j to interval $[t, t + t_j]$.
  $s_j \leftarrow t$; $f_j \leftarrow t + t_j$
  $t \leftarrow t + t_j$
RETURN intervals $[s_1, f_1]$, $[s_2, f_2]$, ..., $[s_n, f_n]$.



2.3.4 Key Observations

Observation 1. *There exists an optimal schedule with no idle time.*



Observation 2. *The earliest-deadline-first schedule has no idle time.*
Definition 7 (Inversion)**.** *Given a schedule S (with jobs numbered so that $d_1 \leq d_2 \leq \cdots \leq d_n$), an* inversion *is a pair of jobs i and j such that $i < j$ but j is scheduled before i.*
Observation 3. *The earliest-deadline-first schedule has no inversions.*
Observation 4. *If a schedule (with no idle time) has an inversion, it has one with a pair of inverted jobs scheduled consecutively.*

2.3.5 Exchange Argument

Claim 1. *Swapping two adjacent, **inverted** jobs reduces the number of inversions by one and does not increase the maximum lateness.*

Proof. Let ℓ be the lateness before the swap, and ℓ' after.
Let $i < j$ be the inverted pair (so $d_i \leq d_j$). For all $k \neq i, j$, we have $\ell'_k = \ell_k$. Job i now finishes earlier, so $\ell'_i \leq \ell_i$. For job j, if it is late:

$$\ell'_j = f'_j - d_j = f_i - d_j \leq f_i - d_i \leq \ell_i,$$

where the first inequality uses $d_i \leq d_j$ (since $i < j$). So the maximum lateness does not increase. □

2.3.6 Proof of Optimality

Theorem 3. *The earliest-deadline-first schedule is optimal.*

Proof (by contradiction). Let S^* be an optimal schedule with the fewest number of inversions. We can assume S^* has no idle time. If S^* has no inversions, then $S^* = S_{\text{EDF}}$ and we are done. Otherwise, let i–j be an adjacent inversion in S^*. Swapping i and j does not increase the maximum lateness and strictly decreases the number of inversions. This contradicts the assumption that S^* has the fewest inversions among optimal schedules. □

2.4 Greedy Analysis Strategies

We have seen three main techniques for proving that a greedy algorithm is optimal:

1. **Greedy stays ahead.** Show that after each step, the greedy solution is at least as good as any other solution.

   Example: Interval scheduling (earliest-finish-time-first).

2. **Structural bound.** Identify a simple lower bound that every solution must satisfy, then show that the greedy algorithm achieves this bound.

   Example: Interval partitioning (depth = lower bound on number of classrooms).

3. **Exchange argument.** Gradually transform any optimal solution into the greedy solution by making small changes ("exchanges") that do not worsen the objective.

   Examples: Minimizing lateness (swapping inversions), Optimal caching (changing eviction decisions).

Other famous greedy algorithms: Gale–Shapley (stable matching), Kruskal and Prim (minimum spanning tree), Dijkstra (shortest paths), Huffman (data compression), and many more.

Chapter 3

Dynamic Programming

3.1 Algorithmic Paradigms

There are three main design strategies for algorithms:

1. **Greedy.** Build a solution step by step, always making the locally best choice at each step.
2. **Divide-and-conquer.** Split the problem into *independent* subproblems, solve each one, and combine the results.
3. **Dynamic programming.** Split the problem into *overlapping* subproblems, solve each one once, and store the result for later reuse.

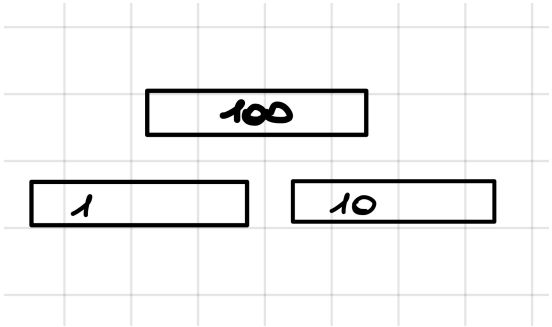
Dynamic programming is essentially a fancy name for *caching intermediate results in a table* so they do not have to be recomputed. The key difference from divide-and-conquer is that subproblems overlap: the same subproblem appears in many branches of the recursion.

3.2 Weighted Interval Scheduling

3.2.1 Problem Definition

Input. A set of n jobs. Job j starts at time s_j, finishes at f_j, and has value (weight) $v_j > 0$. Two jobs are *compatible* if they do not overlap.

Goal. Find a maximum-weight subset of mutually compatible jobs.



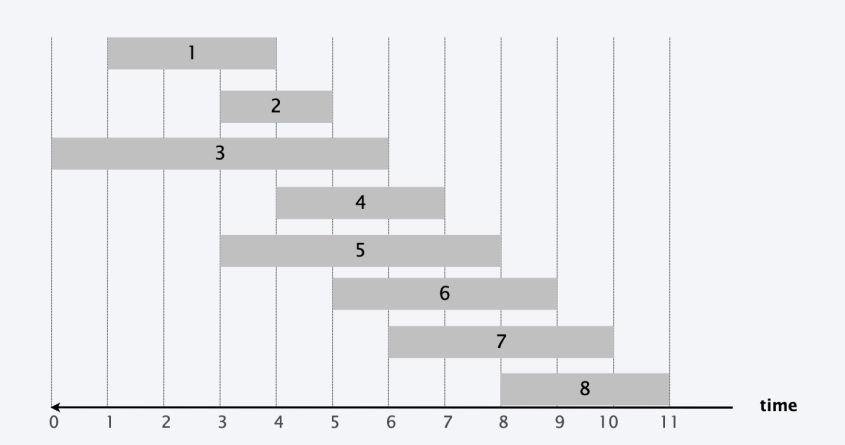
Why greedy fails. Counterexample showing that earliest-finish-time-first fails when jobs have weights because a single high-weight job (e.g. v=100) that overlaps with many low-weight jobs will be ignored by greedy, even though it is clearly the best choice.

3.2.2 Key Definitions

Label jobs by finishing time: $f_1 \leq f_2 \leq \cdots \leq f_n$.

Definition 8 (Compatible predecessor $p(j)$)**.** *$p(j)$ is the largest index $i < j$ such that job i is compatible with job j (i.e., $f_i \leq s_j$). If no such job exists, $p(j) = 0$.*

Example. For a set of 8 jobs: $p(8) = 5$, $p(7) = 3$, $p(2) = 0$.



3.2.3 Optimal Substructure (Binary Choice)

Definition 9. *OPT(j) = value of the optimal solution using only jobs $1, 2, \ldots, j$.*

For the last job j, there are exactly two cases:

1. **OPT does not select job j.** Then $\mathrm{OPT}(j) = \mathrm{OPT}(j-1)$.
2. **OPT selects job j.** Then we collect profit v_j and can no longer use jobs $\{p(j)+1, \ldots, j-1\}$. We must solve optimally for jobs $1, \ldots, p(j)$. So $\mathrm{OPT}(j) = v_j + \mathrm{OPT}(p(j))$.

$$\mathrm{OPT}(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\big\{v_j + \mathrm{OPT}(p(j)),\ \mathrm{OPT}(j-1)\big\} & \text{otherwise} \end{cases}$$

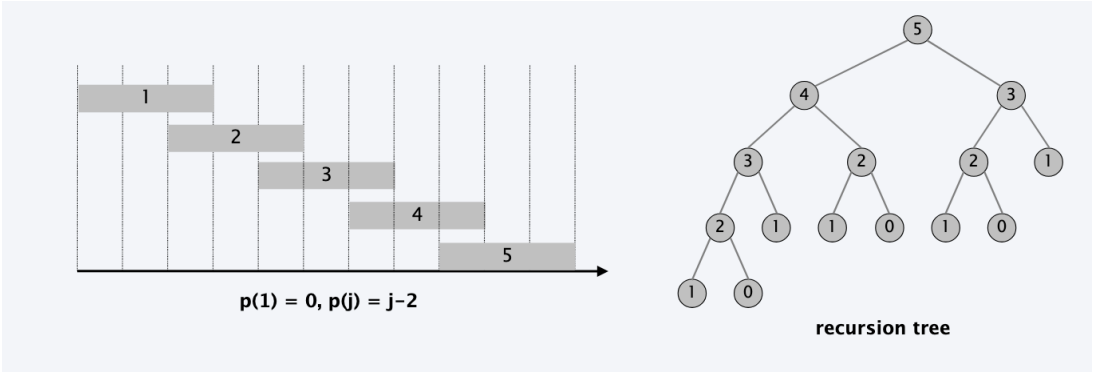
This recurrence has the *optimal substructure property* (proved via exchange argument). This is a **binary choice**: either take job j or leave it.

3.2.4 Brute Force

```
Input:  n, s[1..n], f[1..n], v[1..n]
Sort jobs by finish time so that f[1] ≤ f[2] ≤ … ≤ f[n].
Compute p[1], p[2], …, p[n].

Compute-Opt(j)
if j = 0
   return 0.
else
   return max(v[j] + Compute-Opt(p[j]), Compute-Opt(j-1)).
```

A naive recursive implementation recomputes the same subproblems many times. For a

In pratica, $p(j) = j - 2$ significa che il lavoro j è compatibile solo con il lavoro $j - 2$ (mentre $j - 1$ si sovrappone a j e non può essere scelto insieme a j).Questo rende la ricorsione simile a quella dei numeri di Fibonacci, perché ogni soluzione dipende da due sottoproblemi distinti. n = 5, $p(5) = 3$ significa che il lavoro 5 è compatibile solo con il lavoro 3 (e non con il lavoro 4). Quindi, quando consideriamo se includere il lavoro 5, dobbiamo guardare alla soluzione ottimale per i primi 3 lavori (OPT(3)) invece che per i primi 4 lavori (OPT(4)). In questo caso, OPT(5) dipende da OPT(3) e OPT(4), e OPT(4) dipende da OPT(2) e OPT(3), e così via, creando una struttura ricorsiva che porta a molte chiamate ripetute degli stessi sottoproblemi.



3.2.5 Memoization: Top-Down DP

Key idea. Cache the result of each subproblem the first time it is computed, and look it up directly on future calls.

```
Input:  n, s[1..n], f[1..n], v[1..n]
Sort jobs by finish time so that f[1] ≤ f[2] ≤ … ≤ f[n].
Compute p[1], p[2], …, p[n].

for j = 1 to  n
   M[j] ← empty.
M[0] ← 0.

M-Compute-Opt(j)
if M[j] is empty
   M[j] ← max(v[j] + M-Compute-Opt(p[j]), M-Compute-Opt(j - 1)).
return M[j].
```

Claim 2. *The memoized algorithm runs in $O(n \log n)$ time.*

Proof. Sorting by finish time: $O(n \log n)$. Computing $p(\cdot)$: $O(n \log n)$ via binary search after sorting by start time. Each call to M-Compute-Opt(j) either (i) returns an existing value in $O(1)$, or (ii) fills in a new entry and makes two recursive calls. Let Φ = number of non-empty entries in M: initially $\Phi = 0$, and case (ii) increases Φ by 1. Since $\Phi \leq n$ throughout, there are at most $2n$ recursive calls of type (ii), so M-Compute-Opt(n) uses $O(n)$ time after preprocessing. □

3.2.6 Finding the Actual Solution

The DP computes the optimal *value*. To recover the actual *set of jobs*, do a second pass:

```
Find-Solution(j)
if j = 0
   return ∅.
else if (v[j] + M[p[j]] > M[j-1])
   return {j } ∪ Find-Solution(p[j]).
else
   return Find-Solution(j-1).
```

At most n recursive calls, so $O(n)$ time.

3.2.7 Bottom-Up DP

Instead of recursion with memoization (top-down), we can fill the table iteratively from small to large (bottom-up):

BOTTOM-UP $(n, s_1, \ldots, s_n, f_1, \ldots, f_n, v_1, \ldots, v_n)$

Sort jobs by finish time so that $f_1 \leq f_2 \leq \ldots \leq f_n$.

Compute $p(1), p(2), \ldots, p(n)$.

$M[0] \leftarrow 0$.

FOR j = 1 TO n

$M[j] \leftarrow \max \{ v_j + M[p(j)], M[j-1] \}$.

Bottom-up and top-down (with memoization) produce the same table. The choice between them is largely a matter of personal style and the specific problem.

3.3 Segmented Least Squares

3.3.1 Background: Least Squares

Given n points $(x_1, y_1), \ldots, (x_n, y_n)$ in the plane, the **least squares** line $y = ax + b$ minimises the sum of squared errors:

$$\text{SSE} = \sum_{i=1}^{n} (y_i - ax_i - b)^2.$$

Calculus gives the closed-form solution:

$$a = \frac{n \sum_i x_i y_i - (\sum_i x_i)(\sum_i y_i)}{n \sum_i x_i^2 - (\sum_i x_i)^2}, \qquad b = \frac{\sum_i y_i - a \sum_i x_i}{n}.$$

3.3.2 Problem Definition

In practice, data often follows *several* line segments, not a single line.

Input. n points $(x_1, y_1), \ldots, (x_n, y_n)$ with $x_1 < x_2 < \cdots < x_n$, and a penalty constant $c > 0$.

Goal. Find a sequence of line segments that minimises:

$$f = E + cL,$$

where E is the total sum of squared errors across all segments, and L is the number of segments. The constant c controls the trade-off between *accuracy* (small E) and *parsimony* (small L).

3.3.3 Optimal Substructure (Multiway Choice)

Definition 10.
- $e(i, j)$ = *minimum sum of squared errors for points* $p_i, p_{i+1}, \ldots, p_j$ *(the cost of fitting a single line to this segment).*
- $OPT(j)$ = *minimum cost for points* $p_1, p_2, \ldots, p_j$.

The last segment must end at p_j and start at some p_i. For each choice of i:

$$\text{cost} = e(i, j) + c + \text{OPT}(i - 1).$$

This gives the recurrence:

$$\text{OPT}(j) = \begin{cases} 0 & \text{if } j = 0 \\ \min\limits_{1 \le i \le j} \big\{ e(i, j) + c + \text{OPT}(i - 1) \big\} & \text{otherwise} \end{cases}$$

This is a **multiway choice**: we choose *which point* to start the last segment from, not just a binary yes/no.

Example

Consider four points sorted by their x-coordinates:

$$p_1 = (1, 1), \quad p_2 = (2, 2), \quad p_3 = (3, 2), \quad p_4 = (4, 4).$$

Let the penalty for each segment be $c = 1$.

Assume the following precomputed least-squares errors:

$$e_{1,1} = 0,\ e_{1,2} = 0.5,\ e_{1,3} = 1.2,\ e_{1,4} = 3.0,$$

$$e_{2,2} = 0,\ e_{2,3} = 0.2,\ e_{2,4} = 1.5,$$

$$e_{3,3} = 0,\ e_{3,4} = 0.4, \qquad e_{4,4} = 0.$$

We define $OPT(j)$ as the minimum cost of approximating the first j points. The dynamic programming recurrence is

$$OPT(j) = \min_{1 \le i \le j} \big(e_{i,j} + c + OPT(i - 1) \big), \qquad OPT(0) = 0.$$

Now compute the values:

$$OPT(1) = e_{1,1} + c + OPT(0) = 0 + 1 + 0 = 1,$$

$$OPT(2) = \min\{e_{1,2} + 1 + OPT(0),\ e_{2,2} + 1 + OPT(1)\} = \min\{1.5,\ 2\} = 1.5,$$

$$OPT(3) = \min\{e_{1,3}{+}1{+}OPT(0),\ e_{2,3}{+}1{+}OPT(1),\ e_{3,3}{+}1{+}OPT(2)\} = \min\{2.2,\ 2.2,\ 2.5\} = 2.2,$$

$$OPT(4) = \min\{e_{1,4}{+}1{+}OPT(0),\ e_{2,4}{+}1{+}OPT(1),\ e_{3,4}{+}1{+}OPT(2),\ e_{4,4}{+}1{+}OPT(3)\}$$

$$= \min\{4.0,\ 3.5,\ 3.9,\ 3.2\} = 3.2.$$

Therefore, the optimal cost for all four points is $OPT(4) = 3.2$.

3.3.4 Algorithm

SEGMENTED-LEAST-SQUARES $(n,\ p_1,\ \ldots,\ p_n\ ,\ c)$

FOR j = 1 TO n

  FOR i = 1 TO j

    Compute the least squares $e(i, j)$ for the segment $p_i, p_{i+1}, \ldots, p_j$.

$M[0] \leftarrow 0.$

FOR $j = 1$ TO n

  $M[j] \leftarrow \min_{1 \le i \le j} \{ e_{ij} + c + M[i-1] \}.$

RETURN $M[n]$.

Theorem 4 (Bellman 1961)**.** *The DP algorithm solves the segmented least squares problem in $O(n^3)$ time and $O(n^2)$ space.*

Proof. There are $O(n^2)$ pairs (i, j). Computing $e(i, j)$ for each pair takes $O(n)$ time using the closed-form formula. The bottleneck is therefore $O(n^2) \times O(n) = O(n^3)$. □

By precomputing prefix sums $\sum x_i$, $\sum y_i$, $\sum x_i y_i$, $\sum x_i^2$, each $e(i, j)$ can be computed in $O(1)$, reducing the total to $O(n^2)$ time and $O(n)$ space.

3.4 Knapsack Problem

3.4.1 Problem Definition

Input. n items, a knapsack of capacity W. Item i has weight $w_i > 0$ and value $v_i > 0$.

Goal. Choose a subset of items with total weight $\le W$ that maximises total value.

Example (weight limit $W = 11$):

Item i	Value v_i	Weight w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

Optimal: items $\{3, 4\}$ with value $18 + 22 = 40$. **Greedy strategies** (by value, by weight, by ratio v_i/w_i) all fail to find this.

3.4.2 Why One Variable Is Not Enough

the problem is NP-complete, so we do not expect a polynomial-time algorithm in the input size. However, we can solve it in pseudo-polynomial time using dynamic programming. A first attempt defines $\mathrm{OPT}(i) =$ max profit from items $1, \ldots, i$. This does not work because when we add item i, we do not know the remaining capacity. We need to track *both* which items we have considered *and* how much capacity we have left.

3.4.3 Optimal Substructure (New Variable)

Definition 11. *$OPT(i, w) =$ maximum value from items $1, \ldots, i$ with weight limit w.*

For item i:

1. **OPT does not take item i:** use best of items $1, \ldots, i-1$ with limit w.
2. **OPT takes item i:** gain v_i, reduce capacity to $w - w_i$, and use best of items $1, \ldots, i-1$ with new limit.

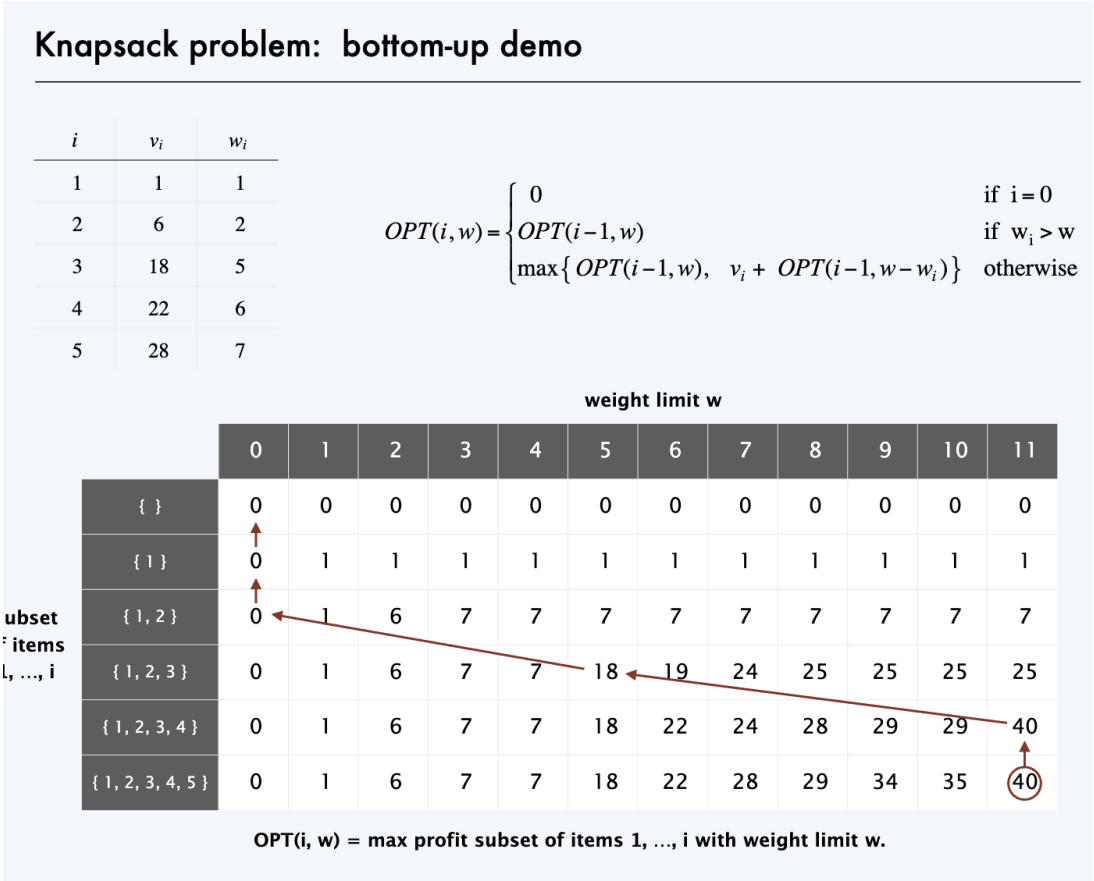
$$\mathrm{OPT}(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ \mathrm{OPT}(i-1, w) & \text{if } w_i > w \\ \max\{\mathrm{OPT}(i-1, w),\ v_i + \mathrm{OPT}(i-1, w-w_i)\} & \text{otherwise} \end{cases}$$

in the second case, we reject item i because it is too heavy. In the third case, we have a binary choice: either reject item i and keep the same capacity, or accept item i, gain its value, and reduce the capacity accordingly. This is the **"adding a new variable"** technique: when one variable is not enough to capture all necessary information, add another.

3.4.4 Bottom-Up Algorithm

```
KNAPSACK (n, W, w1, ..., wn, v1, ..., vn)

FOR w = 0 TO W
    M[0, w] ← 0.

FOR i = 1 TO n
    FOR w = 1 TO W
    IF (wi > w)  M[i, w] ← M[i−1, w].
    ELSE         M[i, w] ← max { M[i−1, w], vi + M[i−1, w−wi] }.

RETURN M[n, W].
```

Tracing the solution. After computing the table, item i is in the optimal solution for (i, w) if and only if $M[i, w] > M[i-1, w]$.



i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

subset of items 1, ..., i	0	1	2	3	4	5	6	7	8	9	10	11
{ }	0	0	0	0	0	0	0	0	0	0	0	0
{ 1 }	0	1	1	1	1	1	1	1	1	1	1	1
{ 1, 2 }	0	1	6	7	7	7	7	7	7	7	7	7
{ 1, 2, 3 }	0	1	6	7	7	18	19	24	25	25	25	25
{ 1, 2, 3, 4 }	0	1	6	7	7	18	22	24	28	29	29	40
{ 1, 2, 3, 4, 5 }	0	1	6	7	7	18	22	28	29	34	35	40

3.4.5 Running Time and Remarks

Theorem 5. *The knapsack DP algorithm runs in $\Theta(nW)$ time and $\Theta(nW)$ space.*

Proof. Each of the $\Theta(nW)$ table entries is filled in $O(1)$ time. □

Important remark. The running time $\Theta(nW)$ is *not* polynomial in the input size. The input size is $O(n \log W)$ (since W is encoded in binary), so the algorithm is **pseudo-polynomial**. In fact:

- The decision version of knapsack is **NP-complete** (Chapter 8).
- There exists a polynomial-time approximation scheme (PTAS) that finds a solution within 1% of the optimum (Section 11.8).

in particular: $if(v_i < V - -w_i < W)$
$I(n,w) = \sum_{i=1}^{n} logw_i + logv_i + logW == O(n * max(logW, logV)) = O(nlogW)$

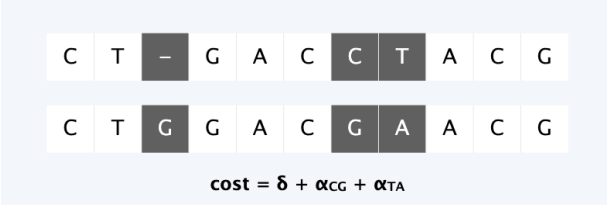
3.5 Sequence Alignment

3.5.1 String Similarity and Edit Distance

A natural question in computer science and biology is: *how similar are two strings?* Consider the strings `ocurrance` and `occurrence`. Depending on how we align them, we get very different costs:

The **edit distance** (Levenshtein 1966, Needleman–Wunsch 1970) measures this cost formally using two parameters:

- δ = gap penalty (cost of leaving a character unmatched)
- α_{pq} = mismatch penalty for aligning character p with character q (equals 0 if $p = q$)



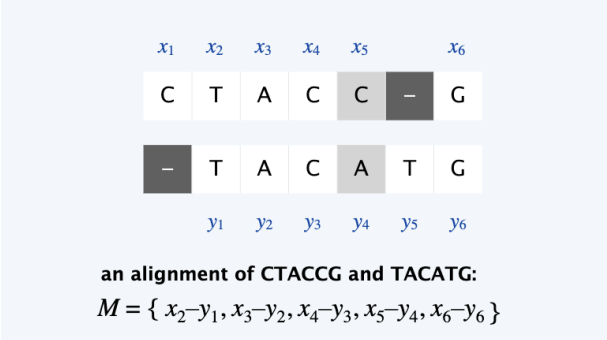
Applications: Unix `diff`, spell checking, speech recognition, computational biology (DNA/protein alignment), and more.

3.5.2 Problem Definition

Definition 12 (Alignment)**.** *Given strings $x_1x_2\dots x_m$ and $y_1y_2\dots y_n$, an* alignment M *is a set of ordered pairs (x_i, y_j) such that each character appears in at most one pair, and there are no* crossings *(i.e., if $(x_i, y_j) \in M$ and $(x_{i'}, y_{j'}) \in M$ with $i < i'$, then $j < j'$).*
Definition 13 (Cost of an alignment)**.**

$$cost(M) = \underbrace{\sum_{(x_i,y_j)\in M} \alpha_{x_i y_j}}_{mismatch} + \underbrace{\delta \sum_{i:\, x_i \text{ unmatched}} 1 + \delta \sum_{j:\, y_j \text{ unmatched}} 1}_{gaps}$$

Goal. Find the alignment of minimum cost.



3.5.3 Optimal Substructure

Definition 14. *OPT*(i, j) = *minimum cost of aligning the prefix strings* $x_1 \dots x_i$ *and* $y_1 \dots y_j$.

There are three cases for what happens to the last characters x_i and y_j:

1. x_i **is matched with** y_j**:** pay mismatch $\alpha_{x_i y_j}$, then solve the subproblem on $x_1 \dots x_{i-1}$ and $y_1 \dots y_{j-1}$.
2. x_i **is left unmatched (gap):** pay δ, then solve on $x_1 \dots x_{i-1}$ and $y_1 \dots y_j$.
3. y_j **is left unmatched (gap):** pay δ, then solve on $x_1 \dots x_i$ and $y_1 \dots y_{j-1}$.

This gives the recurrence:

$$\mathrm{OPT}(i,j) = \begin{cases} j\delta & \text{if } i = 0 \\ i\delta & \text{if } j = 0 \\ \min\left\{\alpha_{x_i y_j} + \mathrm{OPT}(i-1, j-1),\ \delta + \mathrm{OPT}(i-1, j),\ \delta + \mathrm{OPT}(i, j-1)\right\} & \text{otherwise} \end{cases}$$

in the first two cases, we have to pay a gap penalty for each unmatched character because one of the 2 strings is empty. In the third case, we have a **three-way choice**: we can match x_i with y_j, or leave x_i unmatched, or leave y_j unmatched.

3.5.4 Algorithm

SEQUENCE-ALIGNMENT $(m, n, x_1, \dots, x_m, y_1, \dots, y_n, \delta, \alpha)$

FOR $i = 0$ TO m

  $M[i, 0] \leftarrow i\,\delta$.

FOR $j = 0$ TO n

  $M[0, j] \leftarrow j\,\delta$.

FOR i = 1 TO m

  FOR j = 1 TO n

    $M[i,j] \leftarrow \min\ \{\ \alpha[x_i, y_j] + M[i-1, j-1],$

    $\delta + M[i-1, j],$

    $\delta + M[i, j-1])$.

RETURN $M[m, n]$.

Theorem 6. *The DP algorithm computes the edit distance of two strings of length m and n in $\Theta(mn)$ time and $\Theta(mn)$ space.*

Recovering the alignment. To find not just the cost but also the actual alignment, trace back through the DP table from $M[m, n]$ to $M[0, 0]$, following the choice that was made at each step.

Space optimisation. We can compute the optimal *value* in $O(mn)$ time and only $O(m + n)$ space by keeping just two rows of the table at a time. However, this makes it hard to recover the actual alignment.

3.6 Hirschberg's Algorithm

3.6.1 Goal: Linear Space with Full Alignment

Theorem 7. *There exists an algorithm to find an optimal alignment in $O(mn)$ time and $O(m + n)$ space.*

This is achieved by a clever combination of divide-and-conquer and dynamic programming (Hirschberg, 1975). The key insight is to model the problem as a shortest-path problem.

Edit Distance Graph

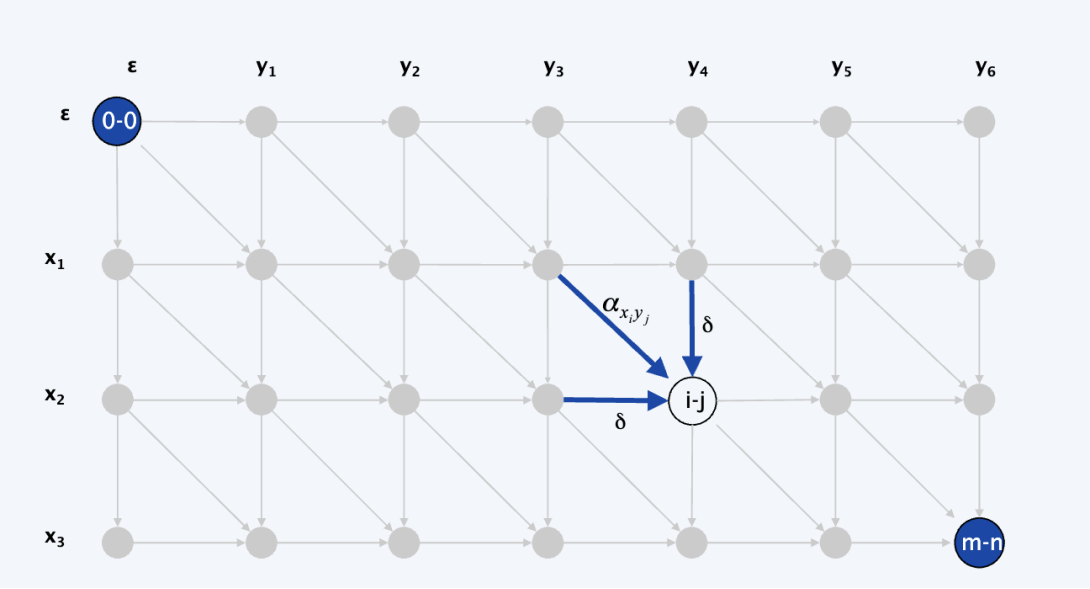


Figure 3.1: The edit distance problem can be represented as a shortest path problem in a directed acyclic graph. Each node corresponds to a pair of indices (i, j), and edges represent the three possible operations (match/mismatch(diagonale), gap in x, gap in y(verticale/orizzontale)) with their associated costs. The optimal alignment corresponds to the shortest path from $(0, 0)$ to (m, n).

Lemma 1. *Let $f(i, j)$ be the cost of the shortest path from $(0, 0)$ to (i, j). Then $f(i, j) = OPT(i, j)$ for all i, j.*

Proof (by strong induction on $i+j$). Base case: $f(0,0) = \mathrm{OPT}(0,0) = 0$. Inductive step: the last edge into (i,j) comes from $(i{-}1,j{-}1)$, $(i{-}1,j)$, or $(i,j{-}1)$. Applying the inductive hypothesis to each predecessor:

$$f(i,j) = \min\{\alpha_{x_i y_j} + f(i{-}1,j{-}1),\ \delta + f(i{-}1,j),\ \delta + f(i,j{-}1)\} = \mathrm{OPT}(i,j). \qquad \square$$

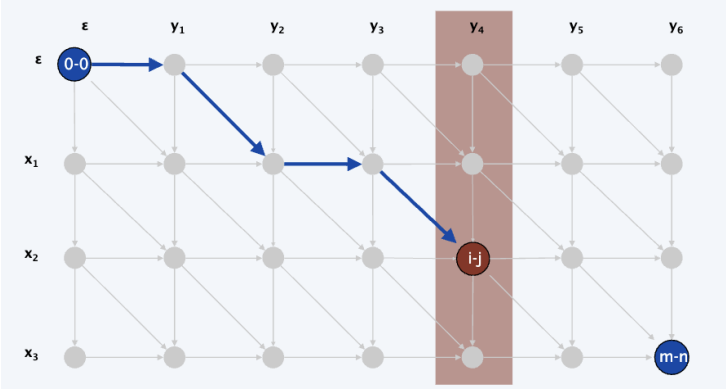
$\square$

Forward and Backward Distances

Define two functions:

- $f(i,j)$ = shortest path from $(0,0)$ to (i,j) (computed *forward*).
- $g(i,j)$ = shortest path from (i,j) to (m,n) (computed *backward*, by reversing all edges and swapping the roles of $(0,0)$ and (m,n)).

Both $f(\cdot,j)$ and $g(\cdot,j)$ can be computed for any fixed column j in $O(mn)$ time and $O(m+n)$ space.



Key observations:

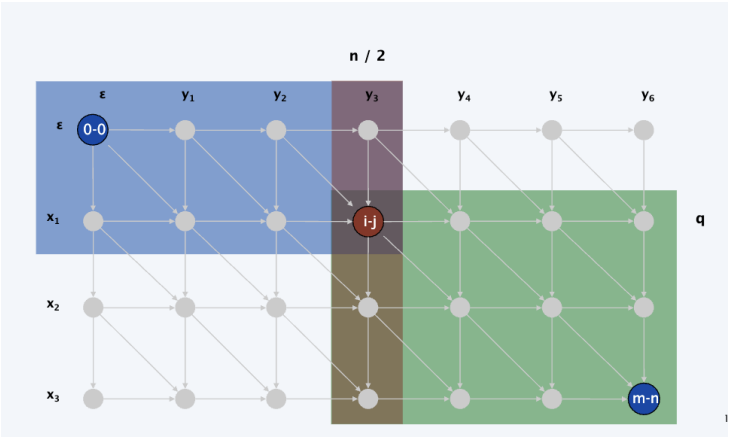
1. The total cost of the shortest path through node (i,j) is $f(i,j) + g(i,j)$.
2. Let q be the index that minimises $f(q, \lfloor n/2 \rfloor) + g(q, \lfloor n/2 \rfloor)$. Then the optimal alignment passes through node $(q, \lfloor n/2 \rfloor)$.

In other words, the optimal alignment is not just any path through the graph: it must pass through a specific node in the middle column. This key observation makes the divide-and-conquer approach possible, because it guarantees that some node in the middle column belongs to the optimal solution, and we can identify it by combining the forward and backward dynamic programming computations.

3.6.2 Divide-and-Conquer

1. **Divide.** Compute $f(\cdot, n/2)$ (forward) and $g(\cdot, n/2)$ (backward). Find index q minimising $f(q, n/2) + g(q, n/2)$. The optimal alignment passes through $(q, n/2)$.
2. **Conquer.** Recursively compute the optimal alignment in:
   - the top-left subproblem: $x_1 \dots x_q$ vs. $y_1 \dots y_{n/2}$

- the bottom-right subproblem: $x_{q+1} \dots x_m$ vs. $y_{n/2+1} \dots y_n$



Example of Hirschberg's Algorithm

Consider the strings $x =$ `AC` and $y =$ `AGC`.

The goal is to find an optimal alignment between these two strings. Hirschberg's algorithm first splits the second string in the middle. Since y has length 3, the middle column is $\lfloor 3/2 \rfloor = 1$, so we look at the first character `A`.

Now we do two computations:

- a forward computation from the start to the middle column,
- a backward computation from the end to the middle column.

For each possible row q, we add the forward cost and the backward cost. The best value tells us which node in the middle column belongs to the optimal alignment.

In this example, the best path passes through the node that matches the first `A`. After that, the problem is split into two smaller parts:

- the left part, before the middle column,
- the right part, after the middle column.

This example shows the main idea of Hirschberg's algorithm: find one correct point in the middle, then solve the two halves recursively. The final result is the same optimal alignment as the standard dynamic programming algorithm, but with much less space.

3.6.3 Running Time Analysis

Theorem 8. *Hirschberg's algorithm runs in $O(mn)$ time and $O(m+n)$ space.*

Proof (by induction on n). Let $T(m, n)$ be the running time. The divide step takes $O(mn)$ time. The two subproblems are of sizes $(q, n/2)$ and $(m-q, n/2)$. Choose constant

c such that the divide step costs at most cmn. Claim: $T(m,n) \leq 2cmn$.

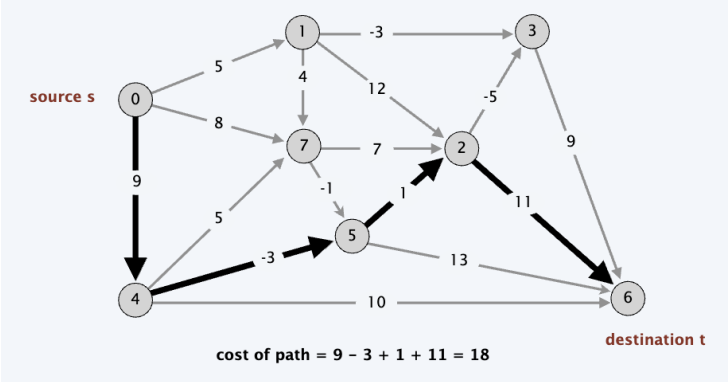
$$\begin{aligned} T(m,n) &\leq T(q,n/2) + T(m-q,n/2) + cmn \\ &\leq 2cq(n/2) + 2c(m-q)(n/2) + cmn \\ &= cqn + cmn - cqn + cmn \\ &= 2cmn. \qquad \square \end{aligned}$$

The key point is that the two subproblems split the *rows* (q and $m-q$), so the total work across all levels of recursion stays $O(mn)$. $\square$

3.7 Bellman-Ford Algorithm

3.7.1 The Shortest Path Problem with Negative Weights

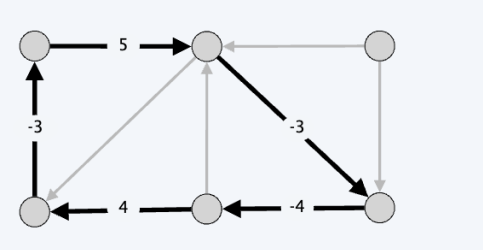
Problem. Given a directed graph $G = (V, E)$ with arbitrary edge weights c_{vw}, find the cheapest path from every node v to a destination node t.



Why Dijkstra fails. Dijkstra's algorithm requires non-negative edge weights. With negative weights it can give wrong answers. A simple fix (adding a constant to all edges) also fails because it changes the lengths of paths with different numbers of edges.

3.7.2 Negative Cycles

Definition 15 (Negative cycle)**.** *A* negative cycle *is a directed cycle W such that $\sum_{e \in W} c_e <$* 0.



Lemma 2. *If some path from v to t contains a negative cycle, then there is no cheapest path from v to t (the cost can be made arbitrarily negative).*

Lemma 3. *If G has no negative cycles, then there exists a cheapest $v \rightsquigarrow t$ path that is simple (uses at most $n-1$ edges).*

Proof. Take a cheapest path with the fewest edges. If it contains a cycle W, removing W gives a cheaper or equal-cost path with fewer edges (since $c(W) \geq 0$). Contradiction. □

Shortest path and negative cycle problems

- Shortest path problem. Given a digraph G = (V, E) with edge weights cvw and no negative cycles, find cheapest v to t path for each node v.
- Negative cycle problem. Given a digraph G = (V, E) with edge weights cvw, determine whether G contains a negative cycle.

The two problems are closely related: if we can solve the shortest path problem, we can check for negative cycles by seeing if any path cost can be reduced by going around a cycle. Conversely,if we can solve the negative cycle problem, we can find shortest paths by first checking for negative cycles and then applying a shortest path algorithm that assumes no negative cycles.

3.7.3 Dynamic Programming Formulation

Definition 16. *$OPT(i, v)$ = cost of the cheapest $v \rightsquigarrow t$ path using at most i edges.*

Recurrence: (è come se partissi da t e facessi passi indietro, ogni passo indietro può essere o restare fermi o prendere un arco in ingresso)

$$\mathrm{OPT}(i,v) = \begin{cases} 0 & \text{if } i = 0 \text{ and } v = t \\ \infty & \text{if } i = 0 \text{ and } v \neq t \\ \min\{\mathrm{OPT}(i-1,v),\ \min_{(v,w)\in E}\{\mathrm{OPT}(i-1,w) + c_{vw}\}\} & \text{otherwise} \end{cases}$$

The two cases are: (1) the cheapest path uses $\leq i-1$ edges (do not move yet), or (2) it uses exactly i edges (take the first edge (v, w) and then find the best $w \rightsquigarrow t$ path with $\leq i-1$ edges).

Key observation. If G has no negative cycles, then $\mathrm{OPT}(n-1, v)$ gives the cost of the cheapest $v \rightsquigarrow t$ path (by Lemma 2, the cheapest path is simple and has at most $n-1$ edges).

3.7.4 Implementation

```
SHORTEST-PATHS(V, E, c, t)
  FOR each node v in V:
      M[0, v] = infinity
  M[0, t] = 0
  FOR i = 1 TO n-1:
      FOR each node v in V:
          M[i, v] = M[i-1, v]
          FOR each edge (v, w) in E:
              M[i, v] = min(M[i, v], M[i-1, w] + c(v,w))
  RETURN M
```

Theorem 9. *Given a digraph with no negative cycles, the DP algorithm computes the cost of the cheapest $v \rightsquigarrow t$ path for each node v in $\Theta(mn)$ time and $\Theta(n^2)$ space.*

3.7.5 Bellman-Ford: Efficient Implementation

In practice, we use only $O(n)$ space by maintaining a single array $d(v)$ (the best distance found so far) and a successor pointer.

```
BELLMAN–FORD(V, E, c, t)
  FOR each node v in V:
      d(v) = infinity
      successor(v) = null
  d(t) = 0
  FOR i = 1 TO n−1:
      FOR each node w in V:
          IF d(w) was updated in previous iteration:
              FOR each edge (v, w) in E:
                  IF d(v) > d(w) + c(v,w):
                      d(v) = d(w) + c(v,w)
                      successor(v) = w
      IF no d(w) value changed in this iteration: STOP
```

Performance optimisation. If $d(w)$ was not updated in iteration $i-1$, there is no reason to consider edges entering w in iteration i. This can make the algorithm much faster in practice.

3.7.6 Analysis and Correctness

Lemma 4. *Throughout Bellman-Ford, $d(v)$ is the cost of* some $v \rightsquigarrow t$ *path. After pass i, $d(v) \leq OPT(i, v)$.*

Proof (by induction on i). After pass $i+1$, for any $v \rightsquigarrow t$ path P with $i+1$ edges, let (v, w) be the first edge and P' be the subpath from w to t. By the inductive hypothesis, $d(w) \leq c(P')$. After considering v in pass $i+1$:

$$d(v) \leq c_{vw} + d(w) \leq c_{vw} + c(P') = c(P). \qquad \square$$

$\square$

Theorem 10. *Given a digraph with no negative cycles, Bellman-Ford computes the costs of the cheapest $v \rightsquigarrow t$ paths in $O(mn)$ time and $\Theta(n)$ extra space.*

Caution. The claim that "following successor pointers always gives a path of cost $d(v)$" is *false* in general during the algorithm (the successor graph may contain cycles or have inconsistent costs). However, upon termination with no negative cycles, the successor graph is a tree and following pointers from any s gives the correct shortest path.

Lemma 5. *If the successor graph contains a directed cycle W at any point during Bellman-Ford, then W is a negative cycle.*

Proof. When `successor(v) = w` is set, we have $d(v) \geq d(w) + c_{vw}$. Let $v_1 \to v_2 \to \cdots \to v_k \to v_1$ be the cycle. Summing the inequalities (with a strict inequality for the last edge

added to the cycle) gives:

$$c(v_1, v_2) + c(v_2, v_3) + \cdots + c(v_k, v_1) < 0. \qquad \square$$

$\square$

Theorem 11. *Given a digraph with no negative cycles, Bellman-Ford finds the cheapest $s \rightsquigarrow t$ paths in $O(mn)$ time and $\Theta(n)$ extra space.*

Proof. By Lemma 4, the successor graph has no cycle on termination. Following successor pointers from s gives a path $s = v_1 \to v_2 \to \cdots \to v_k = t$. Upon termination, for each $\texttt{successor}(v) = w$, we have $d(v) = d(w) + c_{vw}$ (since values no longer change). Summing along the path: $d(s) = d(t) + \sum_{\text{edges}} c = c(P)$. Combined with Theorem 2, $c(P)$ is the minimum cost. $\square$ $\square$

3.8 Distance Vector Protocols

3.8.1 Routing in Communication Networks

In a communication network, nodes are routers, edges are direct links, and edge weights represent link delays. We want each router to know the shortest path to every other router.

Dijkstra needs global knowledge of the whole network. **Bellman-Ford** needs only local knowledge of neighbouring nodes — each router only talks to its neighbours.

3.8.2 Distance Vector Protocols

Each router maintains:

- A vector of shortest known distances to every other node.
- The first hop (direction) on each shortest path.

The algorithm runs n separate Bellman-Ford computations, one for each destination. Updates propagate through the network by *"routing by rumour"*: each router shares its distance vector with its neighbours, and they update their own vectors accordingly.

Examples: RIP, Cisco's IGRP, AppleTalk's RTMP, Novell's IPX RIP.

Asynchrony. The algorithm still converges even if routers do not update in lockstep.

3.8.3 Counting-to-Infinity Problem

Problem. If a link is deleted, distance vector protocols can get stuck in a loop where routers keep increasing their estimated distances, very slowly converging.

Example. If the link $s \to t$ (cost 1) is deleted, s thinks it can reach t via v (cost 2), then v updates via s (cost 3), and so on — slowly counting to infinity.

3.8.4 Path Vector Protocols

Link-state routing fixes the counting-to-infinity problem by storing the full path (not just the distance and first hop). This allows routers to detect loops immediately. Requires much more storage. Used in **BGP** (internet routing) and **OSPF**.

3.9 Negative Cycle Detection

3.9.1 The Problem

Goal. Given a directed graph $G = (V, E)$ with edge weights c_{vw}, find a negative cycle if one exists.

Application: Currency Arbitrage. Given n currencies and exchange rates between pairs, is there a sequence of conversions that makes a profit?

Example. USD $\to$ EUR $\to$ GBP $\to$ USD with rates $0.741 \times 1.366 \times 0.995 = 1.00714 > 1$: this is an arbitrage opportunity. Modelling this as a graph problem (with $-\log$(exchange rate) as edge weights), arbitrage corresponds to a negative cycle.

3.9.2 Detection via Bellman-Ford

Lemma 6. *If $OPT(n, v) = OPT(n-1, v)$ for all v, then there is no negative cycle.*
Lemma 7. *If $OPT(n, v) < OPT(n-1, v)$ for some node v, then the cheapest $v \rightsquigarrow t$ path has exactly n edges. By the pigeonhole principle, it contains a directed cycle W. Since removing W would give a shorter path, W must be a negative cycle.*

3.9.3 Algorithm

Theorem 12. *A negative cycle can be found in $\Theta(mn)$ time and $\Theta(n^2)$ space.*

Proof. Add a new node t and connect every node to t with a 0-cost edge. The new graph G' has a negative cycle if and only if G has one. Run the DP for n iterations (instead of $n-1$). If $\mathrm{OPT}(n, v) = \mathrm{OPT}(n-1, v)$ for all v: no negative cycle. Otherwise, extract the directed cycle from the path from v to t (the cycle cannot contain t since no edges leave t). □

Theorem 13. *A negative cycle can be found in $O(mn)$ time and $O(n)$ extra space.*

Proof. Run Bellman-Ford for n passes (not $n-1$) on the modified digraph G'.

- If no $d(v)$ is updated in pass n: no negative cycle.
- Otherwise, suppose $d(s)$ is updated in pass n. Define pass(v) = last pass in which $d(v)$ was updated. We have pass(s) $= n$ and pass(successor(v)) $\geq$ pass(v) $- 1$. Following successor pointers from s, we must eventually revisit a node. By Lemma 4, the resulting cycle is a negative cycle. □

□

3.9.4 Summary of Complexities

Algorithm	**Problem**	**Time**	**Space**
Sequence Alignment (basic DP)	Edit distance + alignment	$\Theta(mn)$	$\Theta(mn)$
Sequence Alignment (space opt.)	Edit distance only	$O(mn)$	$O(m+n)$
Hirschberg's algorithm	Edit distance + alignment	$O(mn)$	$O(m+n)$
Shortest Paths (basic DP)	No negative cycles	$\Theta(mn)$	$\Theta(n^2)$
Bellman-Ford (efficient)	No negative cycles	$O(mn)$	$\Theta(n)$
Negative Cycle Detection	Find neg. cycle	$\Theta(mn)$ / $O(mn)$	$\Theta(n^2)$ / $O(n)$

Chapter 4

exercises on dynamic programming

Problem 1 (Max Independent Set on Line Graphs)

Let $G = (V, E)$ be a line graph of n nodes formed by a sequence of nodes $v_1, v_2, \ldots, v_n$ with an edge between v_i and v_j if and only if i and j differs by exactly 1. With every vertex i, we associate a positive integer weight w_i. Recall that a set of nodes is an *independent set* if no two nodes of them are joined by an edge. The goal is to find an independent set of G of maximum total weight.

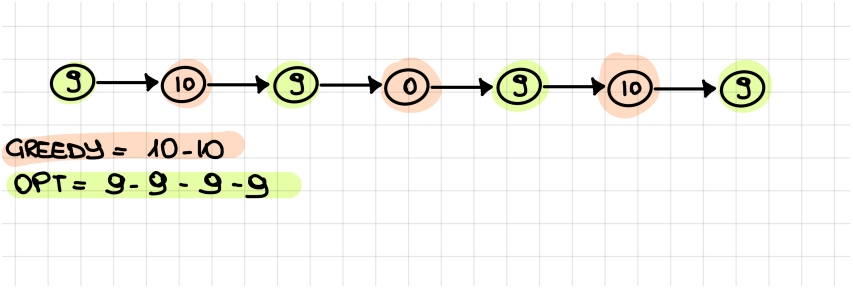
(a) Show on an example that a greedy strategy fails to find an optimal solution.

(b) Give an algorithm that finds an independent set of maximum total weight in time which is polynomial in the number of vertices and it does not depend from the weights of the nodes.

(c) Provide the pseudo-code of an efficient implementation of the algorithm and discuss its time and space complexity.

solution

(a) the greedy strategy fails to find an optimal solution. For example, consider the line graph with 7 nodes and weights $w_1 = 9$, $w_2 = 10$, $w_3 = 9$, $w_4 = 0$, $w_5 = 9$, $w_6 = 10$, $w_7 = 9$. The greedy strategy would pick the greater value so it would pick v_2 and v_6 with total weight 20. However, the optimal solution is to pick v_1, v_3, v_5, and v_7 with total weight 36.



(b) We can solve this problem using dynamic programming. We define OPT(i) as the maximum total weight of an independent set that can be formed from the first i

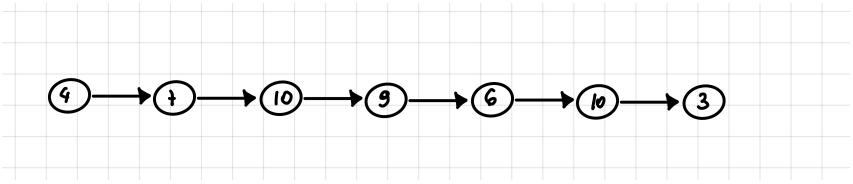
nodes.

$$\mathrm{OPT}(i) = \begin{cases} 0 & \text{se } i = 0 \\ w_1 & \text{se } i = 1 \\ \max\{\mathrm{OPT}(i-1),\ \mathrm{OPT}(i-2) + w_i\} & \text{se } i \geq 2 \end{cases}$$

(c) The pseudo-code of the algorithm is as follows:

```
def max_independent_set(weights):
    n = len(weights)
    OPT[0] = 0
    OPT[1] = weights[1]
    for i =2 to n:
        if weights[i] + OPT[i - 2] > OPT[i - 1]:
            OPT[i] = weights[i] + OPT[i - 2]
        else:
            OPT[i] = OPT[i - 1]
    return OPT[n]
```

example of the algorithm in action:



$$\mathrm{OPT}(0) = 0$$
$$\mathrm{OPT}(1) = 4$$
$$\mathrm{OPT}(2) = \max\{\mathrm{OPT}(1),\ \mathrm{OPT}(0) + 7\} = \max\{4, 7\} = 7$$
$$\mathrm{OPT}(3) = \max\{\mathrm{OPT}(2),\ \mathrm{OPT}(1) + 10\} = \max\{7, 14\} = 14$$
$$\mathrm{OPT}(4) = \max\{\mathrm{OPT}(3),\ \mathrm{OPT}(2) + 9\} = \max\{14, 16\} = 16$$
$$\mathrm{OPT}(5) = \max\{\mathrm{OPT}(4),\ \mathrm{OPT}(3) + 6\} = \max\{16, 20\} = 20$$
$$\mathrm{OPT}(6) = \max\{\mathrm{OPT}(5),\ \mathrm{OPT}(4) + 10\} = \max\{20, 26\} = 26$$
$$\mathrm{OPT}(7) = \max\{\mathrm{OPT}(6),\ \mathrm{OPT}(5) + 3\} = \max\{26, 23\} = 26$$

algorithm to know which nodes are in the optimal solution:

```
def find_solution(OPT, weights, i):
    i = len(weights)
    if i=0:
        return []
    if i=1:
        return [1]
    if weights[i] + OPT[i - 2] > OPT[i - 1]:
        return find_solution(OPT, weights, i - 2) + [i]
```

```
else:
    return find_solution(OPT, weights, i - 1)
```

Problem 2 (Rod Cutting)

We are given a rod of integer length L. We assume that a rod of integer length $0 \leq l \leq L$ has a known value $v(l)$. The problem is to find the best way of cutting the rod, such that the sum of the values of the resulting rods is maximised.

(a) Show a dynamic programming algorithm that solves the problem.

(b) Prove the correctness of the recursive form used, and the optimality of the solution returned by the algorithm.

(c) Provide the pseudo-code of an efficient implementation of the algorithm and discuss its time complexity.

Solution

(a) We can solve this problem using dynamic programming. Let $v(i)$ be the value of a rod of length i, and $R(l)$ be the maximum value obtainable for a rod of length l.

The optimal substructure of the problem allows us to define the following recurrence:

$$R(l) = \begin{cases} 0 & \text{if } l = 0 \\ \max_{1 \leq j \leq l} \{v(j) + R(l-j)\} & \text{if } l > 0 \end{cases}$$

(b) **Proof of Correctness**: An optimal cut for a rod of length l consists of a first piece of length j and an optimal arrangement of the remaining $l - j$ length. Since the value of the first piece $v(j)$ is fixed, to maximize the total value we must maximize the value of the remaining part, which is $R(l-j)$. By iterating over all possible first cuts $j \in \{1, \ldots, l\}$, we are guaranteed to find the global maximum.

v	0	1	1.5	4	1	0.5	0
	0	1	2	3	4	5	6

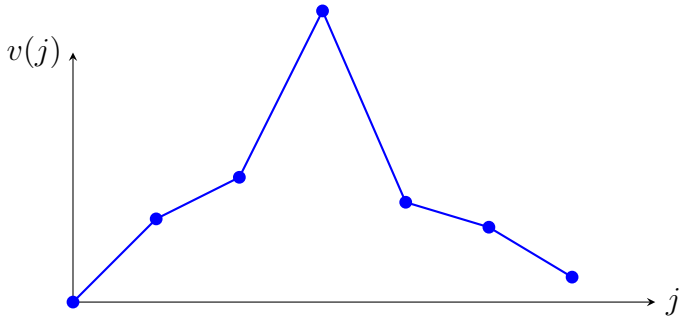


Table $R[l]$ (Max Values)

0	1	2	4	5	6	8
0	1	2	3	4	5	6

$\mathbf{R(0) = 0}$

$\mathbf{R(1) = 1}$

$\mathbf{R(2) = 2}$ $\quad \max\{v(1) + R(1), v(2) + R(0)\}$

Scelta ottima: $j = 1$

$\mathbf{R(3) = 4}$ $\quad \max\{v(1) + R(2), v(2) + R(1), v(3) + R(0)\}$

$v(3) = 4$ $\qquad$ $R(0) = 0$

Scelta ottima: $j = 3$

(c) **Implementation and Complexity**:

First, we consider the basic *bottom-up* implementation that computes only the maximum revenue $R[L]$:

Listing 4.1: Basic Bottom-Up Rod Cutting

```
def rod_cutting(v, L):
    R = [0] * (L + 1)
    for j in range(1, L + 1):
        q = float('-inf')
        for k in range(1, j + 1):
            q = max(q, v[k] + R[j - k])
        R[j] = q
    return R[L]
```

To reconstruct the actual cuts used to achieve this maximum, we extend the algorithm by using an auxiliary array S to store the length of the first piece i that achieved the maximum revenue for each subproblem j:

Listing 4.2: Extended Bottom-Up Rod Cutting

```
def extended_bottom_up_rod_cutting(v, L):
    R = [0] * (L + 1)
    S = [0] * (L + 1)
    for j in range(1, L + 1):
        q = float('-inf')
        for i in range(1, j + 1):
```

```
            if q < v[i] + R[j - i]:
                q = v[i] + R[j - i]
                S[j] = i
        R[j] = q
    return R, S
```

Complexity Analysis:

- **Time Complexity**: $O(L^2)$. The nested loops result in a summation $\sum_{j=1}^{L} j = \frac{L(L+1)}{2}$, which is quadratic.
- **Space Complexity**: $O(L)$ to store the memoization table R and the optimal cuts S.

Solution Reconstruction: To print the lengths of the pieces in an optimal solution for length L:

```
def print_optimal_cuts(S, L):
    while L > 0:
        print(S[L])
        L = L - S[L]
```

Chapter 5

Network Flow

5.1 Max-Flow and Min-Cut Problems

Max-flow and min-cut are two fundamental problems in network flow theory, defined on a directed graph $G = (V, E)$ with a source node s, a sink node t, and a capacity function $c : E \to \mathbb{R}^+$ that assigns a non-negative capacity to each edge.

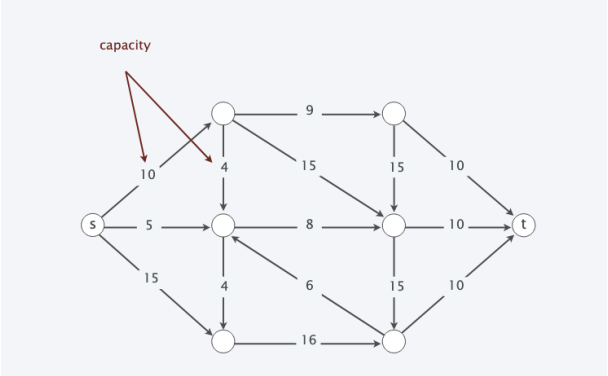


Figure 5.1: Example of a flow network with source s, sink t, and edge capacities.

Definition 17. *An* st-cut *(cut) is a partition* (A, B) *of the vertices such that* $s \in A$ *and* $t \in B$.

Definition 18. *The* capacity *of an st-cut* (A, B) *is the sum of the capacities of edges going* from A to B*:*

$$\text{cap}(A, B) = \sum_{\substack{e=(u,v) \\ u \in A,\, v \in B}} c(e).$$

Edges from B *to* A *are* ***not*** *counted.*

Min-cut problem. Find an st-cut of minimum capacity.

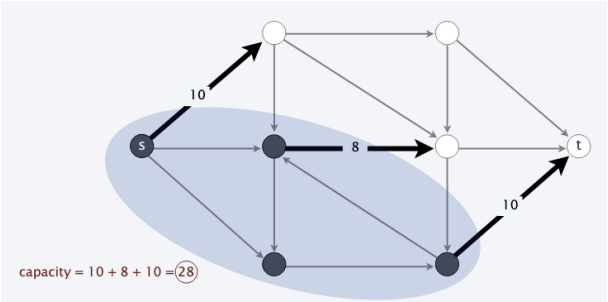


Figure 5.2: capacity of 28.

Definition 19. *An* st-flow *is a function* $f : E \to \mathbb{R}^+$ *satisfying:*

- ***Capacity constraint****: For every edge* $e \in E$, $0 \leq f(e) \leq c(e)$. *un arco non può trasportare più del suo limite di capacità.*
- ***Flow conservation****: For every vertex* $v \in V \setminus \{s, t\}$, *the total flow into* v *equals the total flow out of* v*:*

$$\sum_{\substack{e=(u,v)\\(u,v)\in E}} f(e) = \sum_{\substack{e=(v,w)\\(v,w)\in E}} f(e).$$

un nodo che non è né la sorgente né il pozzo non può accumulare flusso, quindi tutto quello che entra deve uscire.

Definition 20. *The* value *of a flow* f *is*

$$\text{val}(f) = \sum_{e \text{ out of } s} f(e),$$

i.e., the total flow leaving the source s*.*

Max-flow problem. Find an st-flow of maximum value.

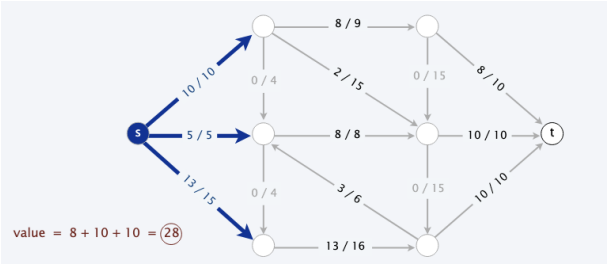


Figure 5.3: Example of a maximum flow with value 28.

5.2 Ford-Fulkerson Algorithm

5.2.1 Greedy Approach and Its Limits

A natural greedy strategy is:

- Start with $f(e) = 0$ for all edges $e \in E$.
- Find an st-path P where every edge e satisfies $f(e) < c(e)$ (i.e., has remaining capacity).

- Push as much flow as possible along P (limited by the minimum remaining capacity on the path).
- Repeat until no such path exists.

Problem with the greedy approach. The order in which augmenting paths are chosen can cause the algorithm to terminate with a sub-optimal flow. For instance, on certain networks the greedy approach reaches a flow of value 16, even though the true maximum is 19. This happens because saturating some edges blocks all remaining paths from s to t, even when additional flow could theoretically be routed differently.

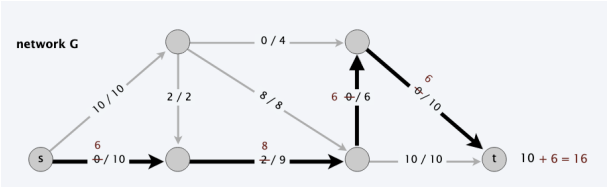


Figure 5.4: Along the augmenting path s→2→4→t, the bottleneck is 8 (limited by edge 2→4, capacity 8), bringing the total flow to 8. Edge s→2 still has $10 - 8 = 2$ units of residual capacity, which are routed along path s→2→3→4→t through edge 2→3 (capacity 2), saturating s→2 and raising the total flow to 10. Finally, the augmenting path s→3→4→5→t has bottleneck 6, yielding a total flow of $10 + 6 = 16$.

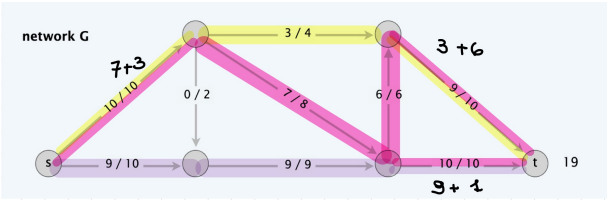


Figure 5.5:

The solution is to use a *residual graph* that allows the algorithm to *undo* previously sent flow, which is the core idea behind Ford-Fulkerson.

5.2.2 Residual Graph

Definition 21. *Given a flow network $G = (V, E)$ and a flow f, the* residual graph *$G_f = (V, E_f)$ is constructed as follows. For each original edge $e = (u, v) \in E$ with capacity $c(e)$ and flow $f(e)$, we create:*

- *A* forward residual edge *$e = (u, v)$ with residual capacity $c_f(e) = c(e) - f(e)$, representing the remaining capacity (included only if $c_f(e) > 0$, i.e., $f(e) < c(e)$).*

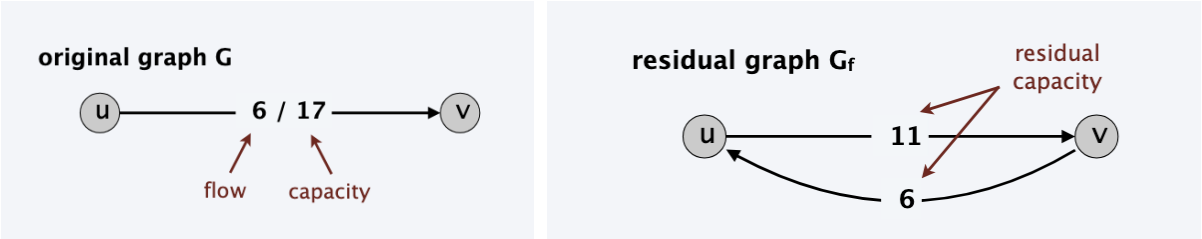


Figure 5.6: The forward residual edge from u to v has capacity $c(e) - f(e)$.

- *A* backward residual edge $e^R = (v, u)$ *with residual capacity* $c_f(e^R) = f(e)$*, representing the flow that can be* cancelled *(included only if* $f(e) > 0$*).*

Formally:

$$E_f = \{e \in E : f(e) < c(e)\} \cup \{e^R : e \in E,\ f(e) > 0\},$$

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \in E, \\ f(e) & \text{if } e^R \in E. \end{cases}$$

Key property. A flow f' in G_f corresponds to a valid flow $f + f'$ in the original graph G.

5.2.3 Augmenting Path

Definition 22. *An* augmenting path *is a simple st-path* P *in the residual graph* G_f *such that every edge on* P *has strictly positive residual capacity.*

Definition 23. *The* bottleneck capacity *of an augmenting path* P *in* G_f *is*

$$\text{bottleneck}(G_f, P) = \min_{e \in P} c_f(e),$$

i.e., the minimum residual capacity among all edges on P*. It is the maximum amount of additional flow that can be pushed along* P *without violating any capacity constraint.*

Key property. Let f be a flow in G and P be an augmenting path in G_f. Let $b = \text{bottleneck}(G_f, P)$. After augmenting along P:

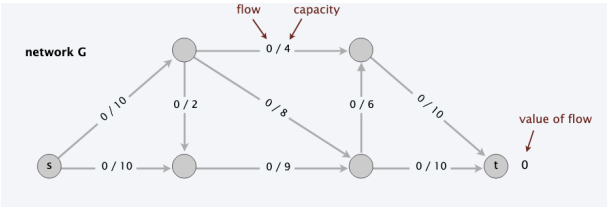
$$\text{val}(f') = \text{val}(f) + b.$$

```
AUGMENT (f, c, P)

b ← bottleneck capacity of path P.
FOREACH edge e ∈ P
    IF (e ∈ E) f(e) ← f(e) + b.
    ELSE       f(e^R) ← f(e^R) − b.
RETURN f.
```

5.2.4 Ford-Fulkerson Algorithm

```
FORD-FULKERSON (G, s, t, c)

FOREACH edge e ∈ E : f(e) ← 0.
   G_f ← residual graph.
   WHILE (there exists an augmenting path P in G_f)
      f ← AUGMENT (f, c, P).
      Update G_f.
   RETURN f.
}
```

example

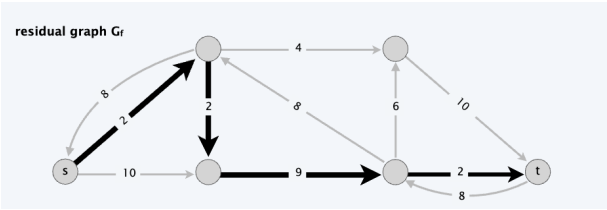


Step 1. Find an augmenting path P_1 in the residual graph G_f. The bottleneck capacity is 8. Augment along P_1:

$$\text{val}(f) = 0 + 8 = 8.$$

Step 2. Find a new augmenting path P_2 in the updated residual graph. The bottleneck capacity is 2. Augment along P_2:

$$\text{val}(f) = 8 + 2 = 10.$$



Step 3. Find augmenting path P_3 with bottleneck capacity 6:

$$\text{val}(f) = 10 + 6 = 16.$$

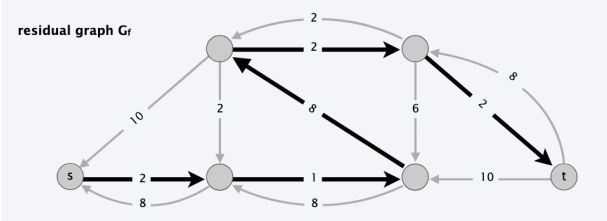
At this point, a purely greedy algorithm (without the residual graph) would get stuck: no forward-edge-only path from s to t remains. However, the residual graph G_f still contains augmenting paths that use **backward edges** to "reroute" flow already sent.

Step 4 — use of a backward edge. In G_f, there exists a path P_4 that traverses a backward (reverse) residual edge. Sending flow along a backward edge *cancels* part of a previously assigned flow, effectively rerouting it. The bottleneck capacity is 2:

$$\text{val}(f) = 16 + 2 = 18.$$

Step 5. One further augmenting path P_5 with bottleneck capacity 1:

$$\text{val}(f) = 18 + 1 = 19.$$



After step 5 there is no augmenting path in G_f, so the algorithm terminates.

5.3 Max-Flow Min-Cut Theorem

The Max-Flow Min-Cut Theorem establishes a fundamental duality between flows and cuts: the maximum value of any st-flow equals the minimum capacity of any st-cut. We build towards this result in three steps.

5.3.1 Flow Value Lemma

Lemma 8 (Flow Value Lemma)**.** *Let f be any st-flow and let (A, B) be any st-cut. Then the* net flow *across (A, B) equals the value of f:*

$$\text{val}(f) = \sum_{\substack{e=(u,v) \\ u \in A,\, v \in B}} f(e) \; - \sum_{\substack{e=(u,v) \\ u \in B,\, v \in A}} f(e).$$

Proof. By definition, $\text{val}(f) = \sum_{e \text{ out of } s} f(e)$. We extend the sum over all nodes in A:

$$\text{val}(f) = \sum_{v \in A} \left(\sum_{e \text{ out of } v} f(e) - \sum_{e \text{ into } v} f(e) \right).$$

By **flow conservation**, every term with $v \neq s$ vanishes. Rearranging the remaining terms by whether the edge crosses the cut ($A \to B$) or goes backwards ($B \to A$) yields the result. □

Edges entirely within A or within B cancel out in the sum, so only edges *crossing* the cut contribute to $\text{val}(f)$.

5.3.2 Weak Duality

Corollary 1 (Weak Duality)**.** *For any st-flow f and any st-cut (A, B):*

$$\text{val}(f) \;\leq\; \text{cap}(A, B).$$

Proof. Using the Flow Value Lemma and the capacity constraint $f(e) \leq c(e)$:

$$\text{val}(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ into } A} f(e) \;\leq\; \sum_{e \text{ out of } A} f(e) \;\leq\; \sum_{e \text{ out of } A} c(e) = \text{cap}(A, B).$$

□

Weak duality immediately implies that any flow value is an *upper bound* on every cut capacity, and vice versa. In particular, if we ever find a flow f and a cut (A, B) such that $\text{val}(f) = \text{cap}(A, B)$, both must be simultaneously optimal.

5.4 Max-Flow Min-Cut Theorem

Theorem 14 (Max-Flow Min-Cut Theorem)**.** *In any flow network, the value of a maximum st-flow equals the capacity of a minimum st-cut:*

$$\max_{f} \text{val}(f) \;=\; \min_{(A,B)} \text{cap}(A, B).$$

Proof. We prove the equivalence of the following three conditions for any flow f:

(i) There exists an st-cut (A, B) such that $\text{cap}(A, B) = \text{val}(f)$.

(ii) f is a maximum flow.

(iii) There is no augmenting path in the residual graph G_f.

(i) ⇒ (ii). Suppose (A, B) is a cut with $\text{cap}(A, B) = \text{val}(f)$. For any other flow f', weak duality gives $\text{val}(f') \leq \text{cap}(A, B) = \text{val}(f)$, so f is a maximum flow.

(ii) ⇒ (iii). We prove the contrapositive. If there exists an augmenting path P in G_f, then we can strictly increase $\text{val}(f)$ by pushing $\text{bottleneck}(G_f, P) > 0$ additional units of flow along P. Hence f is not a maximum flow.

(iii) ⇒ (i). Suppose no augmenting path exists in G_f. Define:

$$A = \{v \in V : v \text{ is reachable from } s \text{ in } G_f\}.$$

By definition, $s \in A$. Since there is no augmenting path, $t \notin A$, so $(A, B = V \setminus A)$ is a valid st-cut.

Now consider any edge $e = (u, v)$ with $u \in A$, $v \in B$:

- The forward residual edge (u, v) cannot be in G_f (otherwise v would be reachable), so $f(e) = c(e)$.
- Any edge $e' = (v, u)$ with $v \in B$, $u \in A$: the backward residual edge (u, v) cannot be in G_f, so $f(e') = 0$.

Therefore, applying the Flow Value Lemma:

$$\text{val}(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ into } A} f(e) = \sum_{e \text{ out of } A} c(e) - 0 = \text{cap}(A, B).$$

□

5.4.1 Augmenting Path Theorem

A direct consequence of the proof above is the following algorithmic certificate:
Corollary 2 (Augmenting Path Theorem)**.** *A flow f is a maximum flow if and only if there is no augmenting path in the residual graph G_f.*

This corollary is the theoretical foundation of the Ford-Fulkerson algorithm: the algorithm terminates precisely when f is optimal, and at that point the set of nodes reachable from s in G_f defines the corresponding minimum cut.

Running Time

Assumption All edge capacities are integers between 1 and C. All edge capacities are integers between 1 and C.
Lemma 9 (Integrality Invariant)**.** *Throughout the Ford-Fulkerson algorithm, all flow values $f(e)$ and all residual capacities $c_f(e)$ remain integers.*

Proof. Initially all flows are 0. Each augmentation increases the flow along a path by bottleneck(G_f, P), which is the minimum of a finite set of integer residual capacities, hence an integer. By induction, all values remain integers throughout. □

Theorem 15. *The Ford-Fulkerson algorithm terminates in at most* $\text{val}(f^*) \leq nC$ *iterations.*

Proof. By the integrality invariant, every augmentation increases val(f) by at least 1. Since val(f^*) is the maximum possible flow value and val$(f^*) \leq nC$ (at most n edges out of s, each with capacity at most C), the algorithm terminates after at most nC iterations. □

Corollary 3. *The running time of Ford-Fulkerson is $O(m \cdot n \cdot C)$, since each iteration requires $O(m)$ time to find an augmenting path via DFS or BFS.*
Corollary 4. *If $C = 1$, the running time of Ford-Fulkerson reduces to $O(mn)$.*
Theorem 16 (Integrality Theorem)**.** *If all edge capacities are integers, then there exists a maximum flow f^* in which every flow value $f^*(e)$ is an integer.*

Proof. Since all capacities are integers, the integrality invariant guarantees that the flow f maintained by the algorithm is always integer-valued. Because the algorithm terminates (by the previous theorem), the flow upon termination is both a maximum flow and integer-valued. □

Ford-Fulkerson is **not** a polynomial-time algorithm in general: its complexity $O(mnC)$ depends on the *value* of the capacities, not just on the *size* of the input. In the worst case, if C is exponentially large, the algorithm takes exponentially many iterations. For polynomial guarantees, variants such as **Edmonds-Karp** (shortest augmenting path, $O(m^2 n)$) or **capacity scaling** ($O(m^2 \log C)$) must be used.

5.5 Capacity-Scaling Algorithm

The basic Ford-Fulkerson algorithm can be very slow if it picks augmenting paths with small bottleneck capacity. The **capacity-scaling algorithm** fixes this by always working with large-capacity paths first.

Key Idea

Instead of using the full residual graph, we work on a restricted subgraph $G_f(\Delta)$ that contains only edges with residual capacity $\geq \Delta$. We start with Δ equal to the largest power of 2 that is $\leq C$ (the maximum capacity), and we halve Δ at each phase.

```
CAPACITY-SCALING(G, s, t, c):
  f(e) = 0 for all edges e
  Delta = largest power of 2 <= C
  WHILE Delta >= 1:
    WHILE there exists augmenting path P in G_f(Delta):
      f = AUGMENT(f, c, P)
      Update G_f(Delta)
    Delta = Delta / 2
  RETURN f
```

Correctness

When $\Delta = 1$, the Δ-residual graph equals the full residual graph. So when the algorithm terminates, there are no augmenting paths in G_f, which means f is a max-flow by the Max-Flow Min-Cut theorem.

Running Time Analysis

- **Lemma 1.** The outer while loop runs at most $1 + \lfloor \log_2 C \rfloor$ times, since Δ starts at most C and halves each time.
- **Lemma 2.** At the end of a Δ-scaling phase, the value of the max-flow satisfies $\text{val}(f^*) \leq \text{val}(f) + 2m\Delta$.
- **Lemma 3.** There are at most $2m$ augmentations per scaling phase. Each augmentation in a Δ-phase increases the flow by at least Δ, and by Lemma 2, at most $2m\Delta$ units remain to be added.

Theorem 17. *The capacity-scaling algorithm finds a max-flow using $O(m \log C)$ augmentations and runs in $O(m^2 \log C)$ time.*

Proof of Lemma 2

At the end of a Δ-scaling phase, let A be the set of nodes reachable from s in $G_f(\Delta)$. Then (A, B) is a cut, and every edge from A to B has residual capacity $< \Delta$, so:

$$\text{val}(f^*) \leq \text{val}(f) + \sum_{e \text{ out of } A} \Delta \leq \text{val}(f) + m\Delta \leq \text{val}(f) + 2m\Delta.$$

5.6 Shortest Augmenting Paths

Another way to improve Ford-Fulkerson is to always pick the augmenting path with the **fewest edges**, found using BFS.

```
SHORTEST-AUGMENTING-PATH(G, s, t, c):
  f(e) = 0 for all edges e
  WHILE there exists augmenting path in G_f:
    P = BFS(G_f, s, t)
    f = AUGMENT(f, c, P)
    Update G_f
  RETURN f
```

Level Graph

The **level graph** L_G of a graph G assigns to each node v a level $\ell(v)$ equal to the number of edges in the shortest path from s to v. It contains only edges (v, w) where $\ell(w) = \ell(v)+1$.

A path is a shortest s-v path in G if and only if it is a path in L_G.

Analysis

- **L1.** The length of the shortest augmenting path never decreases. When we augment along a shortest path, any new edge added to the residual graph goes *backwards*, which can only increase the shortest path length.
- **L2.** After at most m augmentations at the same length, the shortest path strictly increases. Each augmentation saturates at least one edge (the bottleneck), which is then removed from L_G. No new edge is added to L_G until the length increases.

Theorem 18. *The shortest augmenting path algorithm runs in $O(m^2 n)$ time: $O(m+n)$ per BFS, $O(m)$ augmentations at each of the $O(n)$ possible path lengths.*

5.7 Unit-Capacity Simple Networks

A network is **unit-capacity simple** if every edge has capacity 1, and every non-source, non-sink node has either at most one entering edge or at most one leaving edge. Bipartite matching graphs are an important example.

Faster Analysis for This Special Case

Theorem 19 (Even–Tarjan, 1975)**.** *In unit-capacity simple networks, the shortest augmenting path algorithm computes a max-flow in $O(m\sqrt{n})$ time.*

The proof uses three lemmas:

- **L1.** Each phase of normal augmentations takes $O(m)$ time.
- **L2.** After at most $\sqrt{n}$ phases, $f^* - f \leq \sqrt{n}$. After $\sqrt{n}$ phases the shortest path has length $> \sqrt{n}$, so the level graph has more than $\sqrt{n}$ layers. By a pigeonhole argument, some layer V_h has at most $\sqrt{n}$ nodes, giving a cut of capacity $\leq \sqrt{n}$.
- **L3.** After at most $\sqrt{n}$ additional augmentations (one unit each), the flow is optimal.

Combining: $O(\sqrt{n})$ phases $\times$ $O(m)$ per phase + $O(\sqrt{n} \cdot m)$ final augmentations = $O(m\sqrt{n})$.

Application to Bipartite Matching

Since bipartite matching graphs are unit-capacity simple networks, the shortest augmenting path algorithm solves bipartite matching in $O(m\sqrt{n})$ time. This matches the bound of the **Hopcroft–Karp algorithm** (1973).

5.8 Bipartite Matching

A **matching** in a graph $G = (V, E)$ is a subset of edges $M \subseteq E$ such that each node appears in at most one edge. A graph is **bipartite** if the nodes can be split into two sets L and R so that every edge goes from L to R. The goal of bipartite matching is to find a matching of maximum cardinality.

5.8.1 Reduction to Max Flow

Build a directed graph G' as follows:

- Direct all edges from L to R with capacity 1.
- Add a source s with unit capacity edges to every node in L.
- Add a sink t with unit capacity edges from every node in R.

Theorem 20. *The maximum cardinality of a matching in G equals the value of the maximum flow in G'.*

5.8.2 Perfect Matching and Hall's Theorem

A **perfect matching** is a matching where every node appears in exactly one edge. For a bipartite graph with $|L| = |R|$, a perfect matching exists if and only if for every subset $S \subseteq L$, the set of neighbors $N(S)$ satisfies $|N(S)| \geq |S|$. This is **Hall's theorem**.

Theorem 21 (Hall, 1935)**.** *A bipartite graph $G = (L \cup R,\, E)$ with $|L| = |R|$ has a perfect matching if and only if $|N(S)| \geq |S|$ for all $S \subseteq L$.*

As a corollary, every k-regular bipartite graph has a perfect matching, since the flow $f(u, v) = 1/k$ on each edge is a feasible flow of value $|L|$.

5.8.3 Edge-Disjoint Paths

Two paths are **edge-disjoint** if they share no edge. The problem is: given a digraph G and two nodes s and t, find the maximum number of edge-disjoint s-t paths.

5.8.4 Reduction to Max Flow

Assign unit capacity to every edge. Then:

Theorem 22. *The maximum number of edge-disjoint s-t paths equals the value of the maximum flow.*

A direct consequence is **Menger's theorem**: the maximum number of edge-disjoint s-t paths equals the minimum number of edges whose removal disconnects t from s.

For **undirected** graphs, replace each undirected edge with two antiparallel directed edges of capacity 1. There always exists a maximum flow where at most one of each antiparallel pair carries flow, so the same result holds.

5.8.5 Circulation with Demands

A **circulation** is a generalization of flow where each node v has a demand d_v: a positive d_v means v needs to receive d_v more units than it sends, and a negative d_v means v produces flow.

A feasible circulation satisfies:

- **Capacity**: $0 \leq f(e) \leq c_e$ for each edge e.
- **Conservation**: $\sum_{e \text{ into } v} f(e) - \sum_{e \text{ out of } v} f(e) = d_v$ for each node v.

5.8.6 Reduction to Max Flow

Add a super-source s^* and super-sink t^*:

- For each node v with $d_v > 0$, add edge (s^*, v) with capacity d_v.
- For each node v with $d_v < 0$, add edge (v, t^*) with capacity $-d_v$.

A feasible circulation exists if and only if the max flow in this new graph saturates all edges leaving s^*.

5.8.7 Lower Bounds

Edges can also have a **lower bound** ℓ_e: the flow must satisfy $\ell_e \leq f(e) \leq c_e$. This can be reduced to the standard circulation problem by sending ℓ_e units along each edge in advance and updating the demands of both endpoints accordingly.

5.9 Survey Design

A company wants to survey n_1 consumers about n_2 products. Consumer i can only be asked about product j if they own it. Each consumer i must be asked between c_i^- and c_i^+ questions, and each product j must receive between p_j^- and p_j^+ responses.

This is modeled as a **circulation problem with lower bounds**:

- Add an edge (i, j) if consumer i owns product j.
- Add edges from s to each consumer and from each product to t, with lower and upper bounds given by the constraints.
- Add a return edge from t to s.

A feasible integer circulation corresponds to a valid survey design.

5.10 Airline Scheduling

The goal is to manage a set of k flights using the minimum number of crews. A crew can move from flight i to flight j if j is reachable after i ends (same or nearby airport, enough time).

This is modeled as a **circulation problem**:

- For each flight i, add nodes u_i (departure) and v_i (arrival) with edge (u_i, v_i) of lower bound and capacity 1.
- If a crew can go from flight i to flight j, add edge (v_i, u_j) with capacity 1.
- Add source s and sink t to control the total number of crews.

Theorem 23. *The airline scheduling problem can be solved in $O(k^3 \log k)$ time, using binary search on the number of crews and solving $O(\log k)$ circulation problems.*

5.10.1 Project Selection

Each project v has a revenue p_v (which can be negative, representing a cost). Some projects require others: if project v requires project w, then v cannot be chosen without also choosing w. The goal is to find a feasible subset of projects that maximizes total revenue.

Reduction to Min Cut

Build a graph G':

- For each prerequisite edge (v, w), add an edge with ∞ capacity.
- For each project v with $p_v > 0$, add edge (s, v) with capacity p_v.
- For each project v with $p_v < 0$, add edge (v, t) with capacity $-p_v$.

The infinite capacity edges ensure that the selected set is always feasible. The minimum cut in G' gives the maximum revenue subset.